

Pontifícia Universidade Católica de São Paulo
PUC-SP

Wagner Braz Rodrigues

**NoSQL - A análise da modelagem e consistência dos dados na
era do *Big Data*.**

Mestrado em Tecnologias da Inteligência e Design Digital

São Paulo
2017

Wagner Braz Rodrigues

**NoSQL - A análise da modelagem e consistência dos dados na
era do *Big Data*.**

Mestrado em Tecnologias da Inteligência e Design Digital

Dissertação apresentada à Banca Examinadora da Pontifícia Universidade Católica de São Paulo, como exigência parcial para obtenção do título de MESTRE em Tecnologias da Inteligência em Design Digital, sob a orientação do Prof. Dr. Ítalo S. Vega.

São Paulo

2017

Banca Examinadora

A realização deste trabalho foi possível graças à bolsa concedida pela CAPES (Coordenação de Aperfeiçoamento de Pessoal de Nível Superior).

AGRADECIMENTOS

Este trabalho é dedicado de forma geral a todos que me apoiaram e suportaram minha ausência durante sua execução. Agradeço imensamente à minha esposa Fabiana, que aceitou me dividir com meu computador, livros e artigos.

Agradeço ao meu cunhado Fábio Lima, que me auxiliou com a produção da minha proposta de projeto.

Agradeço ao meu orientador Ítalo S. Vega, que me guiou para a execução deste trabalho.

Agradeço aos meus pais, Marisa Romero Braz Peverari e Waldemir Peverari, que me apoiaram para seguir sempre em frente.

Agradeço ao Luiz Fernando, meu filho de quatro patas, que me fez companhia por horas deitado ao pé de minha cadeira.

Agradeço aos meus amigos e companheiros de trabalho, que contribuíram com muitas ideias através de longas conversas.

Agradeço aos meus amigos e companheiros de PUC pelas nossas conversas, apoio mútuo para a conclusão desta etapa em nossas vidas.

Tudo aquilo que o homem ignora, não existe para ele.
Por isso o universo de cada um se resume no tamanho de seu saber.

Albert Einstein

RESUMO

Os novos modelos de armazenamento de dados, conhecidos como NoSQL (*Not Only SQL*), surgem para solucionar as problemáticas de dados atuais, definidas pelas propriedades volume, velocidade e variedade (3 V's) presentes no conceito do *Big Data*. Esses novos modelos de armazenamento se desenvolvem com o suporte da computação distribuída e “escalabilidade horizontal”, o que possibilita o tratamento do grande volume de dados necessários para os V's do *Big Data*. Nesta dissertação é utilizado como referencial teórico o modelo relacional, apresentando suas soluções e problemas. O modelo relacional possibilitou a persistência de estruturas de dados, em memória secundária não volátil. Sua modelagem estabelece regras para a criação de um modelo de dados fundamentado, utilizando conceitos de lógica formal e representação compreensível à interpretação humana. As propriedades definidas pelo modelo transacional ACID (*Atomicity, Consistency, Isolation, Durability*), utilizado em SGBDs (Sistema Gerenciador de Banco de Dados) relacionais, garantem que os dados transacionados serão “persistidos” de maneira consistente na base de dados. O emprego do modelo relacional distanciou as estruturas transientes em memória primária, utilizadas em tempo de execução por aplicações de software e as persistidas em memória secundária, efeito conhecido como “incompatibilidade de impedância”. Os novos modelos apresentados pelas categorias apresentadas no NoSQL trazem estruturas transientes anteriormente utilizadas em memória primária. Contudo, abrem mão da forte estruturação, apresentada pelo modelo relacional. A utilização da computação distribuída apresenta a possibilidade da realização de transações e armazenamento dos dados para vários computadores, conhecidos como nós, presentes em *cluster*. Esse conceito conhecido como tolerância a partição, aumenta a disponibilidade e diminui a possibilidade de falhas em um sistema. No entanto, sua utilização, traz inconsistência aos dados, conforme as propriedades definidas pelo Teorema CAP (FOX; BREWER, 1999). Este trabalho foi realizado através de revisão bibliográfica, analisando primeiramente as necessidades que levaram à criação do modelo relacional. Posteriormente, estabelecemos o estado da arte das teorias e técnicas que envolvem o NoSQL e o tratamento de dados em sistemas distribuídos, bem como as diferentes categorias apresentadas por ele. Foram escolhidas e

analisadas uma ferramenta pertencente a cada categoria de NoSQL para o entendimento de duas estruturas, metamodelos e operações. Além de estabelecer o estado da arte referente ao NoSQL, demonstramos como a reaproximação das estruturas transientes e persistentes se torna possível dado os avanços de máquina atuais, que possibilitaram avanços computacionais, assim como as possibilidades para o tratamento dos efeitos na consistência, demonstrados pelo Teorema CAP.

Palavras-chave: NoSQL, Persistência, Chave-Valor, Documento, Colunar, Grafos, Computação Distribuída.

ABSTRACT

The new storage models, known as NoSQL, arise to solve current data issues, defined by the properties volume, velocity and variety (3 V's) established in the Big Data concept. These new storage models develop with the support of distributed computing and horizontal scalability, which allows the processing of the big amount of data necessary to the Big Data 3 V's. In this thesis was used as theoretical framework the relational model, introducing its solutions and troubles. The relational model allowed the use of structures in secondary memory in a persistent way. Its modeling establishes rules to the creation of a solid data model, using mathematics concepts and tangible representation to the human interpretation. The properties defined by the transactional model ACID, implemented in the relational SGBDs brings assurance consistency of the stored data. The use of the relational model distanced the transient structures in primary memory, used in execution time by software applications and those persisted in secondary memory, an effect known as impedance mismatch. The new models presented by the categories of the NoSQL, bring transient structures previously used in primary memory. The use of distributed computing presents the possibility of the transaction and storage of the data for several computers, known as nodes, present in clusters. Distributed computing increases availability and decreases the likelihood of system failures. However, its use brings inconsistency to the data, according to the properties defined by the CAP Theorem (FOX; BREWER, 1999). This study was carried out on behalf of a bibliographic review, analyzing primarily the needs, which led to the relational model creation. Later, we establish the state of the theoretical and techniques art that involves the NoSQL and the distributed data processing system, just as the different categories introduced by it. An adequate tool were chosen and analyzed from each NoSQL category, for the proper understanding about your structure, metadata and operations. Aside from establish the state of art regarding NoSQL, we demonstrate how the transient and persistent data structures rapprochement becomes possible due to the current machine advances, such as the possibilities to the consistency effect processing, outlined by CAP Theorem

Key words: NoSQL, Persistence, Key-Value, Document, Columnar, Graph, Distributed Computing

LISTA DE FIGURAS

Figura 1 - Diagrama de Venn - Teorema CAP	25
Figura 2 - Arquitetura Cliente-Servidor	29
Figura 3 - Enquadramento arquiteturas CAP	30
Figura 4 - Fontes diferentes	35
Figura 5 - Função de Mapeamento	36
Figura 6 - Fluxo MapReduce	37
Figura 7 - Exemplificação: Chave-Valor	39
Figura 8 - Exemplificação: Colunar	41
Figura 9 - Sete Pontes de Königsberg	43
Figura 10 - Exemplificação: Modelo orientado a grafos	45
Figura 11 - $R=\{A \cup B \cup C\}$	46
Figura 12 - Fluxo modelagem Cassandra	47
Figura 13 - Documento Embutido.....	48
Figura 14 - Documentos Referenciados.....	48
Figura 15 - Exemplificação: NoAM	49
Figura 16 - EER Agregado	50
Figura 17 - Agregação, apenas um conjunto $R = \{A\}$	60

LISTA DE ABREVIATURAS E SIGLAS

API	<i>Application Programming Interface</i>
AQL	<i>Aerospike Query Language</i>
ACID	<i>Atomicity, Consistency, Isolation, Durability</i>
BASE	<i>Basically Available, Soft state, Eventual consistency</i>
BI	<i>Business Intelligence</i>
BSON	<i>Binary JSON</i>
CAP	<i>Consistency, Availability, Partition Tolerance</i>
CASE	<i>Computer-Aided Software Engineering</i>
CQL	<i>Cassandra Query Language</i>
DAAS	<i>Database as a Service</i>
DDD	<i>Domain-Driven Design</i>
ETL	<i>Extract Transform Load</i>
IAAS	<i>Infrasctruture as a Service</i>
GPS	<i>Global Positioning System</i>
JSON	<i>JavaScript Object Notation</i>
LINQ	<i>Language Integrated Query</i>
MER	<i>Modelo Entidade Relacionamento</i>
NLP	<i>Natural Language Processing</i>
NoAM	<i>NoSQL Abstract Model</i>
NXD	<i>Native XML Databases</i>
ODBC	<i>Open Database Connectivity</i>
OLAP	<i>Online Analytical Processing</i>

OLTP	<i>Online Transaction Processing</i>
ODS	<i>Operational Data Store</i>
ORM	<i>Object-Relational Mapping</i>
SGDB	Sistema Gerenciador de Banco de Dados
UDF	<i>User Defined Function</i>
XML	<i>eXtensible Markup Language</i>
XSD	<i>XML Schema Definition</i>
YAML	<i>Ain't Markup Language</i>

LISTA DE SÍMBOLOS

α	Letra grega Alfa
π	Letra grega Pi
σ	Letra grega Sigma
ρ	Letra grega Ro
$\in$	Pertence
$\wedge$	Operador lógico E
$\vee$	Operador lógico OU
$\cup$	União
$\cap$	Interseção
$\rightarrow$	Atribuição
$ $	Tal que
$\Leftrightarrow$	Se e somente se
$\forall$	Para todo

SUMÁRIO

1.	INTRODUÇÃO	18
1.1	Contexto do Trabalho.....	18
1.2	Questão da Pesquisa	20
1.3	Metodologia	21
1.4	Organização do Trabalho	22
2.	Computação Distribuída e Big Data	24
2.1	Computação Cloud	24
2.2	Teorema CAP	25
2.2.1	Consistência	25
2.2.2	Disponibilidade.....	26
2.2.3	Tolerância a Partição.....	26
2.2.4	ACID	26
2.2.5	BASE.....	27
2.2.5.1	Basicamente Disponível	27
2.2.5.2	Estado Leve	28
2.2.5.3	Consistência Eventual	28
2.2.6	Modelos de Consistência	28
2.2.6.1	Consistência Servidor	28
2.2.6.2	Consistência em Memória Primária	30
3.	A Variedade dos Dados.....	32
3.1	Estruturação dos Dados	32
3.1.1	Dados Estruturados	32
3.1.2	Dados Não Estruturados	32
3.1.3	Dados Semiestruturados	33
3.1.3.1	XML (eXtensible Markup Language)	33
3.1.3.2	JSON (JavaScript Object Notation)	33
3.1.3.3	YAML(Ain't Markup Language)	34
3.2	Captura dos Dados	34
3.2.1	Metadados	35
3.2.2	ETL	35
3.2.3	MapReduce	37
4.	Categorias de NoSQL	39

4.1	Chave-valor.....	39
4.2	Colunar	40
4.3	Documentos	42
4.4	Grafos	43
4.5	Modelagem	45
4.5.1	Agregados.....	46
4.5.2	Cassandra	47
4.5.3	Modelo Entidade Relacionamento para Documentos.....	47
4.5.4	Noam	49
4.5.5	Modelagem EER para NoSQL	50
4.6	Comparativo	50
4.6.1	Estruturação	51
4.6.2	Metamodelo.....	51
4.7	Operações Com Dados	52
4.7.1	Aerospike.....	54
4.7.2	Cassandra	55
4.7.3	MongoDB.....	55
4.7.4	Neo4J.....	56
5.	MODELOS DE PERSISTÊNCIA	58
5.1	Persistência Relacional	58
5.1.1	Impedância Objeto-Relacional	59
5.2	Bases Orientadas a Agregação	59
5.3	Bases Orientadas a Grafos	60
5.4	Bancos de dados em Memória Primária	60
6.	CONCLUSÃO	62
6.1	Propostas e Necessidade para Trabalhos Futuros	62
	REFERÊNCIAS	64
	GLOSSÁRIO	70

1. INTRODUÇÃO

1.1 Contexto do Trabalho

O conceito *Big Data* ganhou grande popularidade nos últimos anos sendo que muitas soluções de mercado e pesquisas acadêmicas têm se voltado para este tema. A sua popularidade vem, principalmente, da grande quantidade de dados criados na atualidade. Segundo a União Internacional de Telecomunicações, o tráfego de informações que foi registrado no início de 2016 envolveu 185.000 Gigabits por segundo frente a 30.000 Gigabits no mesmo período de 2008 (SANOU, 2016). Em 2016 foram produzidos por minuto 2.4 milhões de consultas no Google, 150 milhões de e-mails enviados, 69.444 horas de conteúdo assistido na *Netflix* e 20.8 milhões de mensagens enviadas no *Whatsapp* (ZENNARO, 2016).

A crescente da criação de dados decorre de diversos fatores como o expoente do número de usuários em face da inclusão digital e a diminuição no custo da implementação de canais informacionais (LANEY, 2001). Esses dados são gerados por vários domínios de aplicações, como smartphones, web, redes sociais, comércio online e transações bancárias (VIEIRA et al., 2012; LANEY, 2001; SILVA, 2012; VERA et al., 2015). Para melhor entendimento do *Big Data* pode-se utilizar a decomposição realizada por Laney (2001), como os 3 V's: volume, variedade e velocidade.

- Volume: a diminuição do custo para a implementação de canais pela Web permitiu a crescente utilização. O aumento do volume de dados produzidos trouxe a oportunidade da extração de informação destes dados;
- Velocidade: com o grande volume de dados e consumidores para os dados surge a necessidade de tecnologias mais rápidas para garantir a entrega;
- Variedade: a variedade surge de diversas fontes de dados, trazendo diferentes tipos, estruturas e diferentes semânticas, tais como: vídeos, *posts* em *blogs*, documentos;

Alguns trabalhos propõem o crescimento da decomposição de Laney para 5 V's, adicionando veracidade e valor, porém estes itens estão sujeitos à subjetividade e análise do conteúdo dos dados (GIL; SONG, 2016). O *Big Data* está intrinsecamente ligado ao processamento analítico deste grande volume de dados, sendo utilizado por muitos autores como sua definição. Dessa forma, o *Big Data* pode ser resumidamente

definido como o processamento analítico de grandes volumes de dados que são produzidos por várias aplicações (VIEIRA et al., 2012).

O poder computacional da atualidade, com maior processamento e capacidade de armazenamento para os computadores, permite o processamento analítico de grandes volumes de dados para a extração de informações (POSPIECH; FELDEN, 2015). Por conseguinte, neste contexto, definimos o processo analítico como uma série de processos, em um conjunto de dados, cujo objetivo é extrair informações produtivas (CUZZOCREA; SONG; DAVIS, 2011). Estes processos são realizados através de técnicas para análises combinatórias, “mineração de dados” e modelos matemáticos ou estatísticos.

A ideia e desejo da análise de dados para a obtenção de inteligência não é um tópico recente. No clássico “Fundação”, de Isaac Asimov, de 1952, seu personagem Heri Seldon analisava matematicamente a sociedade para prever seus rumos. Por analogia, a análise de dados é utilizada em diversas soluções, bem como para a compreensão dos hábitos e necessidade dos clientes, o que gera vantagens competitivas (POSPIECH; FELDEN, 2015).

O tratamento dos conceitos definidos pelos 3 V's de Laney levam a uma série de problemas computacionais. Para o tratamento deste volume e velocidade de dados são necessárias técnicas de computação distribuídas, que paralelizam as transações efetuadas nos SGBDs (ABELLÓ, 2015). Analogamente, a variedade de fontes dados traz desafios a sua modelagem, pela utilização de dados modelados para outras aplicações, causando inconsistências semânticas e estruturais (SAGIROGLU; SINANC, 2013).

Nos últimos anos, um conjunto de novos SGBDs vem surgindo para armazenamento e tratamento de dados, esses SGBDs são conhecidos como NoSQL. Segundo diversos autores, o surgimento do NoSQL decorre do fato de que SGBDs relacionais não se encaixam corretamente no contexto do *Big Data* (VIEIRA et al., 2012; CUZZOCREA; SONG; DAVIS, 2011; ABELLÓ, 2015; LIMA; MELLO, 2015). Isto decorre por sua forte estruturação, que traz desafios para a variedade do *Big Data* e baixa compatibilidade do ACID para a computação distribuída.

Na linha do tempo, o primeiro registro da utilização do termo NoSQL foi em uma base de dados relacional, o Strozzi NoSQL, criado por Carlo Strozzi (SADALAGE; FOWLER, 2012). Este banco de dados, apesar de relacional, não utilizava SQL como

linguagem de consulta. Porém, a base de Strozzi não tem ligação direta com o uso do termo contemporaneamente, que ganhou notoriedade após ser utilizado como uma *hashtag* #NoSQL para representar uma conferência organizada por Johan Oskarsson, em 2009, na cidade São Francisco (EUA). Em contraste, ao uso do termo na conferência, a denominação do termo como "*Not Only SQL*" foi adotada posteriormente de maneira mercadológica, enquadrando SGBDs que utilizam a linguagem SQL (SADALAGE; FOWLER, 2012).

1.2 Questão da Pesquisa

O modelo relacional é baseado na Lógica de Predicados de primeira ordem, formulado por Codd (1970). O modelo relacional por ele proposto baseia-se na teoria dos conjuntos, plano e produto cartesiano. Existem melhores práticas conhecidas no que diz respeito à concepção de bases de dados relacionais. Tais como a normalização utilizando formas de normais (por exemplo, 3NF ou BCNF), que reduz a redundância garantindo a integridade dos dados (Codd, 1970). Estes fatores trazem ao modelo relacional uma forte estruturação no esquema de dados.

Em complemento ao modelo relacional, Chen propôs o artefato, modelo entidade relacionamento (MER). A construção do modelo relacional utiliza relações matemáticas para expressar a estrutura dos dados e seus valores, enquanto no MER a mesma construção é usada para expressar a estrutura das entidades (Chen, 2015). O artefato de Chen simplificou o entendimento do modelo relacional e formalizou a sua diagramação. O MER possibilitou um melhor entendimento sobre o modelo relacional e propiciou sua popularização.

Na concepção do modelo relacional, Codd almejava que os utilizadores estivessem isentos de conhecer a estrutura interna e protegidos de qualquer alteração realizada na base de dados. Para tal, o modelo relacional fornece uma forma de descrever os dados em sua estrutura natural sem a necessidade de alterações para a interpretação dos mesmos por máquinas (Codd, 1970). Consequentemente, fornece linguagem de alto nível para o tratamento e persistência dos dados.

Codd (1970) define uma base de dados por três componentes básicos: um conjunto de tipos de estrutura de dados, um conjunto de operadores ou regras de inferência e um conjunto de regras de integridade. Porém, várias propostas para base de dados apenas definem as estruturas, por vezes omitindo operadores e/ou regras

de integridade (ANGLES; GUTIERREZ, 2008), como é o caso dos NoSQL. O NoSQL não se restringe a apenas um modelo, assim como o modelo relacional – um conjunto de SGBDs utiliza essa nomenclatura. Estes SGBDs são divididos por sua modelagem, em quatro categorias – chave-valor, colunar, documentos e grafos – que apresentam diferentes modelos para a persistência, consistência e operabilidade dos dados (KAUR; RANI, 2013). As estruturas apresentadas pelas bases NoSQL eram anteriormente comuns para a utilização em modelos transientes, ou seja, em tempo de execução em memória primária, possibilitado pelo poder computacional.

Com as definições do modelo relacional, as estruturas de dados persistentes em disco se tornaram muito diferentes das transientes. As bases apresentadas pelo movimento NoSQL reaproximam estes modelos, o que nos leva ao questionamento: estas diferenças ainda fazem sentido dado o atual cenário da computação distribuída e seu efeito no armazenamento e processamento?

1.3 Metodologia

O NoSQL é relativamente novo e seu início parece ter se dado mais por necessidade corporativa do que por pesquisa e conhecimento do mundo acadêmico (SILVA, 2012). A medida que novas necessidades e soluções para aplicações de software surgem, são criadas novas semânticas para sua descrição e identificação (GARLAN; SHAW, 1993). Dado este cenário, este trabalho será iniciado com o estado da arte sobre os problemas do *Big Data* e o avanço de máquina que tornou possível sua produção e processamento.

Para o estabelecimento do estado da arte foi realizada revisão bibliográfica. Esta revisão utilizou artigos, documentações e trabalhos prévios com foco no tema. Para a realização de testes e entendimento dos diferentes modelos de NoSQL, foram adotados um SGBD por categoria: O Aerospike¹ para o chave-valor, o Cassandra² para o Colunar, o MongoDB³ para o Documentos e o Neo4J⁴ para Grafos. Foram utilizados os bancos de dados de artigos, Google Acadêmico, Capes, *CiteSeerx*, IEEE e ACM. A revisão bibliográfica foi dividida conforme descrito abaixo:

¹ Disponível em: <https://www.aerospike.com>. Acesso em: 01 abr. 2016.

² Disponível em: <http://cassandra.apache.org/>. Acesso em: 01 abr. 2016.

³ Disponível em: <https://www.mongodb.com/>. Acesso em: 01 abr. 2016.

⁴ Disponível em: <https://neo4j.com/product/>. Acesso em: 01 abr. 2016.

- (a) Modelo relacional: realizado o entendimento do estado da arte do modelo relacional para situar quais os problemas que esse modelo visa resolver e quais as limitações que levaram à necessidade de modelos alternativos;
- (b) NoSQL: foram levantadas informações sobre o ecossistema do NoSQL visando descobrir: I) quais as origens e as definições inerentes ao NoSQL; II) quais são os fatores e motivadores para a adoção do NoSQL; III) qual a problemática em que o NoSQL é aplicado em soluções arquiteturais e IV) aprofundando o item III, quais as características que diferenciam as categorias de NoSQL;
- (c) Problemática das soluções de dados: analisando os conceitos definidos pelos V's do *Big Data*, e as soluções computacionais propostas para seu tratamento;
- (d) Estrutura de dados: busca do paralelo entre as categorias de NoSQL, visando o entendimento dos “porquês” do metamodelo que cada categoria implementa;
- (e) Modelagem: quais são as propostas para a realização da modelagem dos NoSQL;

1.4 Organização do Trabalho

Este trabalho está estruturado em cinco capítulos, sendo o primeiro introdutório, apresentando a delimitação da questão pesquisada. O segundo capítulo, intitulado **Computação Distribuída e Big Data**, os motivadores que levaram a criação das bases NoSQL. Entre os motivadores são expostas a questão da computação distribuída, a estruturação dos dados e as demais razões arquiteturais. Este assunto é abordado de forma a elucidar a preterição ao NoSQL do modelo relacional para tratar as problemáticas atuais do mundo dos dados.

O terceiro capítulo, intitulado **Estruturação dos Dados**, apresenta as questões ligadas a variedade dos dados. São explorados, diferentes formatos de dados utilizados para armazenamento e transacionamento, bem como as questões semânticas da utilização de diversas fontes, e técnicas para sua captura.

O terceiro capítulo, intitulado **Categorias de NoSQL**, apresenta as diferentes categorias de NoSQL, explorando os seguintes aspectos: a estruturação do metamodelo, visando a principal diferenciação entre os NoSQL e seu comparativo dentre as estruturas de dados clássicas. Em seguida, são analisadas as operações realizadas para a manutenção dos dados, utilizando como base as operações

primitivas realizadas pelo modelo relacional. E, por fim, são apresentadas algumas propostas de modelagem para o NoSQL.

O quarto capítulo, intitulado **Modelos de Persistência**, aprofunda os conceitos apresentados nos capítulos anteriores, enquadrando-os em um contexto histórico sobre a evolução dos modelos de persistência. O quinto capítulo, **Conclusão**, encerra o trabalho com a conclusão e propostas para trabalhos futuros.

2. Computação Distribuída e *Big Data*

Este capítulo visa expor o estado da arte do *Big Data* e da computação distribuída, elucidando as teorias e necessidades que levaram à criação do NoSQL para seu tratamento.

2.1 Computação *Cloud*

A computação distribuída, é um paradigma computacional que interliga computadores, chamados de “nós”, em conjuntos de um ou mais computadores chamados de *cluster* (TANENBAUM; STEEN, 2006). As tarefas executadas nos nós do *cluster*, são distribuídas de maneira que sua execução é realizada paralelamente.

Para a realização da distribuição computacional, aplicações contemporâneas utilizam o conceito de *cloud*. No *cloud* os recursos computacionais, são abstraídos e entregues em forma de serviços (AGRAWAL; DAS; ABBADI, 2011). No contexto do *Big Data* se fazem necessários os SGBDs como serviços DaaS(*Database as a Service*), e infraestrutura como serviço o IaaS(*Infrastructure as a Service*).

O volume dos dados, bem como a velocidade necessária para atender a demanda das aplicações leva a necessidade de aplicações escaláveis, com alocação dinâmica de recursos. O IaaS permite a alocação de recursos sob demanda dadas as necessidades da aplicação suportando o DaaS (SILVA, 2012). Estes recursos, são alocados, de maneira horizontal, utilizando diversos nós do *cluster*. Este princípio é chamado de escalabilidade horizontal (KAUR; RANI, 2013). Em contrapartida, modelos arquiteturais tradicionais, a escalabilidade é feita de forma vertical adicionando recursos como memória, armazenamento e processadores em um mesmo servidor.

A computação distribuída e *cloud*, tornaram possível, tratar o grande volume de dados e satisfazer as necessidades de velocidade, definidas pelas propriedades do *Big Data*. Contudo, a utilização da computação distribuída tem problemas de compatibilidade, quando utilizadas com SGBDs relacionais. Estas incompatibilidades, são demonstradas através da lógica do Teorema CAP.

2.2 Teorema CAP

O teorema CAP (*Consistency, Availability, Partition Tolerance*) de Brewer restringe as características transacionais em sistemas distribuídos a três propriedades: consistência, disponibilidade e tolerância ao particionamento. A lógica proposta por Brewer em seu teorema, demonstra que a utilização das três propriedades simultaneamente é impossível. Portanto, no máximo duas dessas propriedades podem ser satisfeitas (FOX; BREWER, 1999).

Figura 1 - Diagrama de Venn - Teorema CAP



Fonte: Hewitt (2016).

2.2.1 Consistência

A consistência de dados refere-se ao estado interno do sistema de armazenamento. Ou seja, um sistema se torna consistente no momento em que todas as cópias de um mesmo dado estejam no mesmo estado (VIEIRA et al., 2012). Havendo uma alteração no estado do dado, suas cópias são replicadas em todos os nós do *cluster* para sua atualização (AGRAWAL; DAS; ABBADI, 2011). Desta forma, qualquer nó acessado que contenha uma cópia do dado retornará o mesmo estado. Contudo, a atualização em todos os nós do *cluster*, pode comprometer o desempenho. Conforme a definição do CAP, pode-se abrir mão da forte consistência para a garantia da disponibilidade em um sistema distribuído (VERA et al., 2015). Segundo Vogels (2009), a consistência de um sistema pode

ser calculada utilizando os seguintes parâmetros: N = número de nós que contém as réplicas dos dados; W = número de réplicas que precisa reconhecer o recebimento do dado antes da atualização ser concluída; R = número de réplicas que são consultadas quando um dado é acessado; enquanto o sistema mantiver $R + W > N$ sua consistência estará garantida.

2.2.2 Disponibilidade

A disponibilidade de um sistema, é definida, por sua capacidade de permanecer operacional (VERA et al., 2015). Em um sistema distribuído, a disponibilidade é alcançada através da replicação dos dados efetuada pelos nós presentes no *cluster*. Na ocorrência de falhas em um dos nós, os demais permanecem operacionais (SHARP et al., 2013). O mesmo, não pode ser alcançado em um sistema centralizado, dado que se o único nó do sistema falhar, o sistema pode se tornar inoperante.

2.2.3 Tolerância a Partição

Na presença de uma falha de rede alguns dos nós do sistema podem deixar de ser acessíveis aos outros, tornando o *cluster* particionado (VERA et al., 2015). Em um sistema tolerante a partição, os diferentes nós do *cluster*, continuam operacionais, apesar da perda de mensagens com os demais. Se não for tolerante à partição, em caso de falhas o banco de dados pode se tornar disponível apenas para operações de leitura, ou completamente inutilizável.

Segundo o Teorema CAP, SGBDs relacionais utilizam as propriedades Consistência e Disponibilidade, por utilizarem o modelo transacional ACID.

2.2.4 ACID

Os SGBDs relacionais implementam o modelo transacional ACID (Atomic, Consistent, Isolated, Durable) para realizar suas transações. Segundo Haerder, o ACID pode ser definido como um conjunto de regras que quando implementadas trazem forte consistência para os dados. Esse conjunto de regras é definido da seguinte maneira (HAERDER; REUTER, 1983):

- Atômico: os dados de uma transação são indivisíveis; todos os dados devem ser consistidos na base de dados. Em caso de falhas, nenhum dado será consistido;
- Consistente: o estado do banco de dados deve ser sempre consistente, nenhuma transação pode ferir as regras existentes;
- Isolado: Em caso de concorrência entre transações, as mesmas não podem realizar alterações em um mesmo registro, sendo necessário o enfileiramento para a sua execução;
- Durável: uma vez inserido, o dado deve permanecer na base, suportando qualquer falha eventual;

A consistência total é mais fácil de obter em um ambiente não distribuído, geralmente é suportada pela maioria dos SGBDs que cumprem o ACID (VERA et al., 2015). Consistência e tolerância à Partição podem coexistir, porém, afetam a disponibilidade do sistema. No caso de uma partição em um SGBD relacional, transações para um mesmo banco de dados são bloqueadas até que as partições atinjam o mesmo estado. Evitando o risco de conflitos, que tornariam os dados inconsistentes (FOX; BREWER, 1999). Para possibilitar a tolerância à partição, bases NoSQL utilizam outro modelo transacional, o BASE.

2.2.5 BASE

O modelo transacional BASE, como o próprio nome sugere, é o oposto lógico de ACID (SADALAGE; FOWLER, 2012). Seu nome é o acrônimo de Basicamente Disponível, Estado Leve e Consistência Eventual (SHARP et al., 2013).

2.2.5.1 Basicamente Disponível

Esta propriedade é diretamente relacionada a disponibilidade do teorema CAP, quanto a garantia de operacionalidade. O sistema sempre apresentará uma resposta a uma operação, mesmo que esta apresente falha na obtenção dos dados, ou dados em estado inconsistente (SHARP et al., 2013).

2.2.5.2 Estado Leve

O estado do sistema pode ser alterado, a qualquer instante, por uma nova transação ou pela replicação dos demais nós (FOX; BREWER, 1999). Desta forma, o estado do sistema é sempre considerado "leve".

2.2.5.3 Consistência Eventual

O modelo de consistência eventual descrito pelo BASE define que o sistema será consistente. Isto é, todos os nós do sistema irão "eventualmente" ter o mesmo estado. A consistência eventual é definida por $R + W \leq N$.

O período entre a transação, e o término da replicação das cópias nos demais nós é chamado de janela de inconsistência (VERA et al., 2015). O tamanho desta janela de inconsistência pode ser determinado com base em vários fatores. Dentre eles, o número de transações que estão sendo executadas e o número de réplicas.

2.2.6 Modelos de Consistência

A consistência do sistema é diretamente influenciada pelo modelo arquitetural utilizado. Diferentes modelos, podem ser utilizados para a consistência através de sua infraestrutura, ou pelo tratamento das transações em memória primária (VOGELS, 2009).

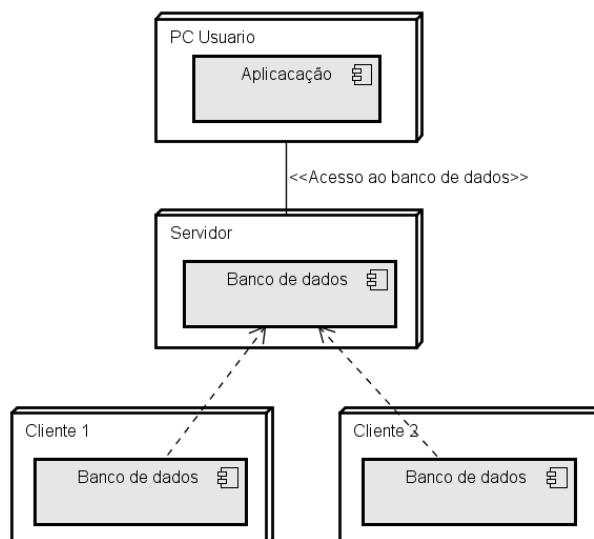
2.2.6.1 Consistência Servidor

A consistência eventual do NoSQL é garantida pela utilização do quórum⁵, que define quantos nós do *cluster* precisam ser comunicados por uma transação, para sua conclusão (KLEIN et al., 2015; AEROSPIKE, 2014). O quórum de escrita é definido por $W > N/2$; o número de nós participantes na escrita W deve ser maior que a metade do número de nós envolvidos na replicação N . Por consequência, a utilização do quórum de consistência para operações de leitura e escrita, garante que a maioria dos nós respondam à leitura com o valor mais recente. O quórum, é

⁵ Quantidade mínima obrigatória de membros presentes ou formalmente representados para que uma assembleia possa deliberar e tomar decisões válidas

comum a ambos os modelos arquiteturais, Cliente-Servidor e Ponto a Ponto, utilizados nos SGBDs NoSQL.

Figura 2 - Arquitetura Cliente-Servidor

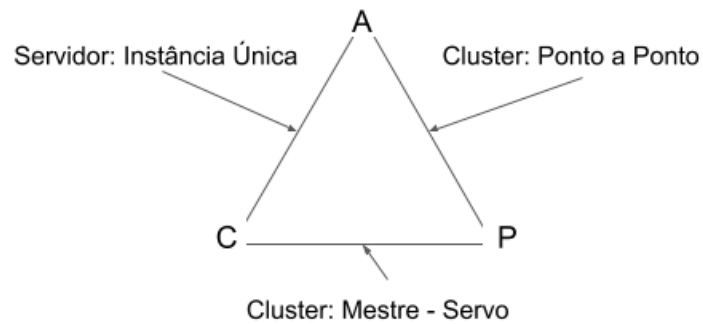


Fonte: Autoria Própria

O esquema Cliente-Servidor definido na Figura 2 utiliza um servidor central para a recepcionar as transações. Uma vez a transação realizada na base, as operações resultantes desta são replicadas a todas as bases clientes, ou a um número determinado (quórum). Cada transação realizada pode especificar o número de servidores em que a escrita deve ser realizada, antes de retornar como bem-sucedida. Para garantir a consistência, a sua operação é realizada somente pelo banco Servidor (MONGODB, 2017b).

Em contrapartida ao esquema Cliente-Servidor, o esquema Ponto a Ponto permite que as transações sejam realizadas em qualquer nó presente no *cluster*. Em sua organização os servidores se coordenam para sincronizar a cópia dos dados em todos os nós (HEWITT, 2016). A operação de escrita é retornada com sucesso assim que o quórum mínimo de servidores é atualizado com os dados da transação. O esquema Ponto a Ponto é descentralizado, o acesso é realizado em qualquer nó presente no *cluster*. Por esta característica, este é utilizado para a obtenção de disponibilidade e tolerância a partição. Portanto, em sua utilização os dados podem ser imprecisos, mas o sistema estará sempre disponível.

Figura 3 - Enquadramento arquiteturas CAP



Fonte: Hewitt (2016).

2.2.6.2 Consistência em Memória Primária

Existem modelos que podem ser adotados pelo desenvolvedor para garantir a consistência na interação da aplicação com a base de dados. Esses são definidos, pelo encadeamento dos processos que realizam escritas e leituras na memória (TANENBAUM; STEEN, 2006).

- **Consistência Causal:** no caso do processo $P1$ comunicar ao processo $P2$ a ocorrência de uma transação, o subsequente acesso do processo $P2$ retornará o valor atualizado (VOGELS, 2009). No caso de uma transação $P3$ sem relação causal com Processo $P3$, o mesmo estará sujeito a consistência eventual, sem garantias de acesso ao valor atualizado.

P1:	W(x1)	R(x1)
P2:	R(x1)	
P3	W(x2)	R(x2)

- **Read Your Writes:** o efeito de uma operação de escrita por um processo em um dado x será sempre visto, por uma operação de leitura subsequente em x pelo mesmo processo.

L1:	W(x1)	
L2:	WS(x1;x2)	R(X2)

O processo realiza a escrita $W(x_1)$ seguida pela operação de leitura $R(x_2)$ é realizada em L2 contendo uma cópia diferente do dado X_2 . A garantia que o processo de leitura irá acessar o dado mais recente é expressa por WS ($x_1; x_2$) que garante que $W(x_1)$ é parte de $W(x_2)$.

- Consistência por sessão: uma variação de *Read Your Writes* onde uma transação sempre acessa a base no contexto de sessão. Enquanto a sessão existir, a sua consistência será garantida (VOGELS, 2009).
- Leituras monotônicas: se um processo lê o valor do dado x , qualquer operação de leitura posterior em x por esse mesmo processo, sempre retornará esse mesmo valor ou um mais valor recente (TANENBAUM; STEEN, 2006).

L1:	$W(x_1)$	$R(x_1)$
L2:	$WS(x_1; x_2)$	$R(x_2)$

O processo de leitura $R(x_1)$ e $R(x_2)$ deve obter a mesma versão do dado X . Esta consistência é garantida quando o conjunto de escritas WS engloba X_1 e X_2 , denotado por WS ($x_1; x_2$).

- Escritas monotônicas: a serialização das escritas realizadas por uma mesma operação (VOGELS, 2009).

L1:	$W(x_1)$
L2:	$WS(x_1) \quad W(x_2)$

Observa-se que a consistência da escrita monotônica é realizada pelo enfileiramento das operações de gravação, por um mesmo processo.

3. A Variedade dos Dados

Este capítulo visa expor, as questões inerentes a utilização da diversidade de formatos e fontes de dados. Demonstrando, diferentes estruturas utilizadas para o transacionamento e armazenamento de dados, e técnicas utilizadas em sua captura.

3.1 Estruturação dos Dados

Segundo Tahara, Diamond e Abadi (2014), as estruturas dos dados podem ser categorizadas em dados estruturados, não estruturados e semiestruturados. Vale ressaltar que esta categorização é focada na comparação com bases de dados relacionais. O suporte a tipos de dados semiestruturados ou não estruturados foi um forte fator para a popularização do NoSQL (VIEIRA et al., 2012).

3.1.1 Dados Estruturados

Dados estruturados, como o nome sugere, são informações armazenadas em campos fixos dentro de um registro ou arquivo (TAHARA; DIAMOND; ABADI, 2014). Bancos de dados relacionais e notações tabulares são exemplos de dados estruturados. São geralmente exibidos em colunas e linhas nomeadas, sendo fácil o acesso por seu chaveamento.

3.1.2 Dados Não Estruturados

Dados não estruturados são dados cujo seu conteúdo não possui analogia direta com o modelo relacional, sendo sua estrutura interna não passível de indexação em SGBDs (TAHARA; DIAMOND; ABADI, 2014). Exemplos populares de dados não estruturados são *e-mails*, áudio, vídeo, *posts* de mídia social, entre outros. Em 2008 foram produzidos 11.430 *petabytes* de dados não estruturados. Em 2015 este número se elevou para 226.716 *petabytes* (OBERAI'S, 2015).

A natureza dos dados não estruturados dificulta a busca, recuperação e análise desses dados e a integração direta com dados estruturados é um desafio. Técnicas avançadas como processamento de linguagem natural são necessárias para converter dados não estruturados em dados estruturados (OBERAI'S, 2015).

3.1.3 Dados Semiestruturados

Os dados semiestruturados são aqueles que não estão em conformidade com a estrutura formal dos bancos de dados relacionais, mas contêm chaves para definir hierarquias de seu conteúdo. Todos os valores armazenados correspondem a uma chave para seu acesso (TAHARA; DIAMOND; ABADI, 2014). Os exemplos mais populares incluem o formato XML, JSON, BSON e YAML. Estes formatos são comumente utilizados para o armazenamento, tráfego e serialização de dados.

3.1.3.1 XML (*eXtensible Markup Language*)

O XML foi desenvolvido pela W3C⁶, na década de 1990 é um dos subtipos da SGML (*Standard Generalized Markup Language*) capaz de descrever diversos tipos de dados. Seu propósito principal é a facilidade de compartilhamento de informações através da internet. Bases de dados com estruturas nativas em XML, as NXD (*Native XML Databases*), surgiram logo após a criação do XML (TOTH, 2008). Entre essas bases temos o MarkLogic⁷ e eXist⁸. Apesar do surgimento destas bases antecederam o movimento NoSQL, diante de sua popularidade elas aderiram a ele.

Exemplo de uma estrutura em XML:

```
< Pessoa >  
  < nome > "Jão Silva" </ nome >  
  < idade > "42" </ idade >  
</ Pessoa >
```

3.1.3.2 JSON (*JavaScript Object Notation*)

O JSON⁹ propõe um formato otimizado utilizando uma notação de declaração de objetos da linguagem *JavaScript*, realizada em pares chave-valor. Por ser uma notação proveniente de uma linguagem de programação, a leitura

⁶ Disponível em: <https://www.w3.org/>. Acesso em: 01 abr. 2017.

⁷ Disponível em: <http://www.marklogic.com/>. Acesso em: 01 mar. 2017.

⁸ Disponível em: <http://www.exist-db.org/exist/apps/homepage/index.html>. Acesso em: 01 mar. 2017.

⁹ Disponível em: <http://www.json.org/json-pt.html>. Acesso em: 01 abr. 2017.

humana é facilitada. O BSON¹⁰, *Binary JSON*, utilizado pelo MongoDB, é uma extensão do JSON codificada em binários. O BSON oferece suporte a tipos de dados adicionais, ordenação de campos, sendo mais facilmente implementável.

Exemplo de uma estrutura em JSON:

```
{
  "pessoa" : {
    "nome" : "Jão Silva ",
    "idade" : "42" }
}
```

3.1.3.3 YAML(*Ain't Markup Language*)

O YAML¹¹ foi criado com o objetivo de realizar a serialização dos dados. O XML foi projetado para ser compatível com o SGML que, por sua vez, foi projeto para oferecer suporte a documentação de estruturas. O XML apresenta restrições de design por seu intuito original ser uma linguagem de marcação como o SGML. O YAML é o resultado do aperfeiçoamento de formatos pregressos como o XML e o JSON. Sua estrutura é similar ao JSON e em sua notação é estruturada por indentação.

Exemplo de uma estrutura em YAML:

```
Pessoa:
  nome: "Jão Silva"
  Idade: "42"
```

3.2 Captura dos Dados

O processo de captura dos dados é definido, pela obtenção de dados produzidos em outras aplicações, para o seu processamento e transacionamento (KIMBALL, 1998). Uma das dificuldades encontradas na realização deste processo, é o entendimento de seus metadados.

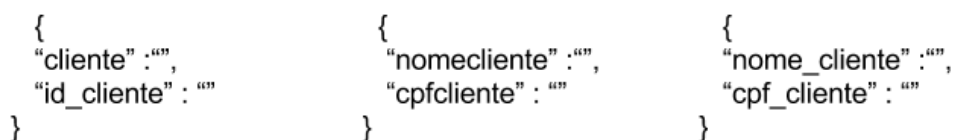
¹⁰ Disponível em: <http://bsonspec.org/>. Acesso em: 01 abr. 2017.

¹¹ Disponível em: <http://yaml.org/>. Acesso em: 01 out. 2016.

3.2.1 Metadados

O prefixo meta vem da língua grega e significa “além de”. Neste contexto, metadados são dados que acrescentam dados aos dados. Os metadados descrevem a estrutura e os significados sobre os dados e assim contribuem para o seu uso (RÊGO, 2013), conforme explicitado na Figura 4, onde demonstramos a captura de clientes de diversas origens em seu formato bruto, ou seja, no mesmo formato de sua origem sem sofrer alterações. Em cada origem vemos o domínio “CPF” e “Nome Cliente” escritos com metadados divergentes.

Figura 4 - Fontes diferentes



```
{
  "cliente" : "",
  "id_cliente" : ""
}
```

```
{
  "nomecliente" : "",
  "cpfcliente" : ""
}
```

```
{
  "nome_cliente" : "",
  "cpf_cliente" : ""
}
```

Fonte: Autoria Própria

Em determinados casos a nomenclatura pode ser críptica, principalmente quando capturados dados de sistemas legados, sem correta documentação (OBERAI'S, 2015). O processo de captura pode até exigir uma investigação no código fonte para o entendimento. A captura destes dados brutos, em seu formato de origem, para transformá-los para seu armazenamento é feita pelo processo de ETL (CUZZOCREA; SONG; DAVIS, 2011).

3.2.2 ETL

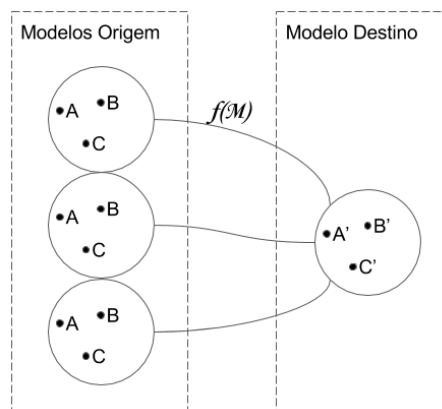
O processo ETL (*Extract-Transform-Load*) tem sido utilizado para a integração de dados de múltiplas fontes ou aplicações de domínios diferentes. Este processo é utilizado para a criação de bases centralizadas como ODS (*Operational Data Store*), armazéns de dados ou camadas para cacheamento (TAHARA; DIAMOND; ABADI, 2014; CUZZOCREA; SONG; DAVIS, 2011). O processo de ETL é constituído das seguintes fases: extração, transformação e carga.

Na realização da fase de extração são selecionados os dados relevantes e extraídos em formato bruto das origens (BANSAL, 2014). Os dados na fonte podem

estar disponíveis em formatos de arquivos simples, como csv, xls e txt ou através de integração sistêmica, por API *RESTful* ou conexão direta ODBC (*Open Database Connectivity*) com outras bases de dados.

A fase de transformação é a mais significativa dentro do processo. Esta fase realiza as transformações necessárias para cumprir as necessidades da base destino (BANSAL, 2014). Dentre estas transformações são realizadas expressões regulares, conversões, filtragem e criação do dicionário de dados. A dicionarização transporta os metadados de sua origem para satisfazer o formato da base destino (TAHARA; DIAMOND; ABADI, 2014), sendo uma função de mapeamento, dada por: $f(M) = \{C1, C2, C3 \dots Cn\} \rightarrow D$.

Figura 5 - Função de Mapeamento



Fonte: Autoria Própria

O processo de ETL é encerrado com a realização da carga, dos dados para a base destino. A baixa estruturação presente em algumas categorias de NoSQL simplifica adaptação do esquema origem para o destino (AGRAWAL; DAS; ABBADI, 2011). Possibilitando o armazenamento de formatos similares ao bruto, para transformações posteriores. Existem soluções que propõem o uso integrado de SGBD Relacional e NoSQL. Em resumo, estas soluções armazenam os dados utilizando arquiteturas NoSQL e utilizam o paradigma *MapReduce* para processos ETL (VIEIRA et al., 2012).

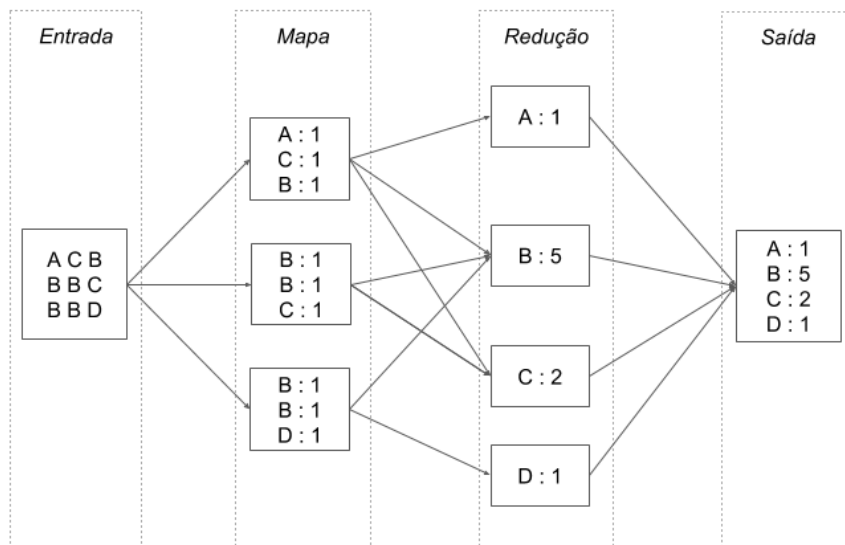
3.2.3 MapReduce

MapReduce é um paradigma de programação para computação distribuída criado pelo Google que quebra os dados em pequenas unidades para o seu processamento em paralelo (SAGIROGLU; SINANC, 2013).

O *MapReduce*, tem como base as funções algorítmicas da programação funcional¹² *Map* e *Reduce* (DEAN; GHEMAWAT, 2008). A função do *Map* associa o mapeamento M a um elemento e pertencente a um determinado conjunto, com o elemento d pertencente ao conjunto dos domínios $M(d) = e$ (AHO ALFRED V.; HOPCROFT, 1983). A função de redução utiliza o mapeamento e realiza uma operação matemática inserida, em todos os valores do conjunto mapeado $M(d) = \sum(e_1, e_2, e_3 \dots e_n)$. O número de réplicas é conhecido como fator de replicação (SADALAGE; FOWLER, 2012).

Um trabalho *MapReduce* divide o conjunto de dados de entrada em pares chave-valor independentes, que são processados pela função de mapeamento (MONGODB, 2017b). A estrutura classifica as saídas dos mapas, que são então inseridos nas tarefas de redução. É uma prática usual o armazenamento da entrada e saída em sistemas de arquivos (MONGODB, 2017b; APACHE, 2013).

Figura 6 - Fluxo *MapReduce*



Fonte: (MONGODB, 2017b).

¹² Paradigma da programação em que a computação é descrita em forma de funções.

Programas escritos neste estilo funcional são automaticamente paralelizados e executados em um *clusters* (APACHE, 2013; DEAN; GHEMAWAT, 2008). Em sua execução, os dados de entrada são particionados em diferentes nós do *cluster*, para a execução das tarefas de forma sequencial e paralela, conforme o fluxo demonstrado na Figura 6. Isto permite que os programadores sem qualquer experiência com sistemas distribuídos facilmente utilizem os seus recursos. Estes processos são úteis para o processamento de grandes quantidades de dados brutos não estruturados e semiestruturados para calcular diversos tipos de derivações (DEAN; GHEMAWAT, 2008).

4. Categorias de NoSQL

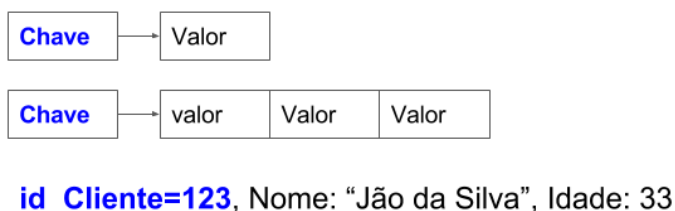
Existe um grande conjunto de SGBDs que se enquadram dentro do NoSQL. Esses divergem principalmente na estruturação e indexação dos dados. As categorias Chave-valor, Colunar, Documentos e Grafos são formas de estruturação de dados derivadas de estruturas clássicas, sendo que algumas destas já foram utilizadas em propostas de bancos de dados anteriormente ao NoSQL.

4.1 Chave-valor

A estrutura de dados dicionário é descrita por Skiena (2008) da seguinte forma: "o tipo de dados do dicionário permite o acesso a itens de dados por conteúdo. Você insere um item em um dicionário para que você possa encontrá-lo quando você precisar dele"(SKIENA, 2008). O acesso aos itens dos dicionários é feito a partir de uma chave que endereça o valor. O acesso realizado a partir da chave do dicionário torna simples a utilização de operações básicas com conjuntos de dados. Estas operações são inserir, localizar e deletar (AHO ALFRED V.; HOPCROFT, 1983).

Os bancos de dados do tipo chave-valor são um exemplo de estrutura de dicionário. Os dados são organizados como uma matriz associativa de entradas, compostas por pares chave-valor. Cada chave é única e é usada para recuperar os valores associados a ela (KAUR; RANI, 2013), conforme exemplificado na Figura 7.

Figura 7 - Exemplificação: Chave-Valor



Fonte: autoria própria.

Bases chave-valor como o Aerospike e Riak-KV¹³ implementam uma forma específica de dicionário, as tabelas HASH (BASHO, 2015). Tabelas HASH são formas muito eficazes de utilização de dicionários. Em sua implementação, a criação da chave para o acesso do valor é obtida através do cálculo do mesmo (CORMEN et al., 2001). A realização do cálculo da chave por meio do valor inserido traz a garantia da unicidade e facilita a indexação.

4.2 Colunar

Em sua definição, matematicamente uma lista é uma sequência de zero ou mais elementos de um determinado tipo (AHO ALFRED V.; HOPCROFT, 1983), sendo uma estrutura básica de dados que permite a composição de estruturas como listas encadeadas.

Bases colunares utilizam as estruturas do tipo listas para a composição das colunas. As chaves das colunas são agrupadas em conjuntos chamados família de colunas que formam uma unidade básica de controle de acesso (CHANG FAY; DEAN, 2006). A Tabela 1 traz um exemplo de uma estrutura de tabular, logo em seguida é demonstrado como os dados são organizados em seu armazenamento.

Tabela 1 – Estrutura Tabular.

id	idpessoa	Nome	Idade
1	123	Jão da Silva	33
2	124	Maria da Silva	31

Fonte: autoria própria.

Estrutura de armazenamento tradicional:

1 : 123 , " Jao da Silva " , 33 ; 2 : 124 , "Maria da Silva " , 31 ;

¹³ Disponível em: <http://basho.com/products/riak-kv/>. Acesso em: 17 fev. 2017.

Listas são comumente utilizadas para a criação de aplicações que realizam a busca de dados (AHO ALFRED V.; HOPCROFT, 1983). Estas buscas são realizadas por varredura: dada uma informação para busca, a lista toda é consultada para encontrar a informação. Este tipo de solução propicia soluções analíticas, consultando todos dados de uma determinada condição, ao invés de buscar diretamente pela chave. Em um armazenamento colunar, cada coluna é armazenada em um arquivo diferente. Dada esta condição, uma operação de varredura em uma coluna será realizada somente no arquivo desta coluna (HEWITT, 2016).

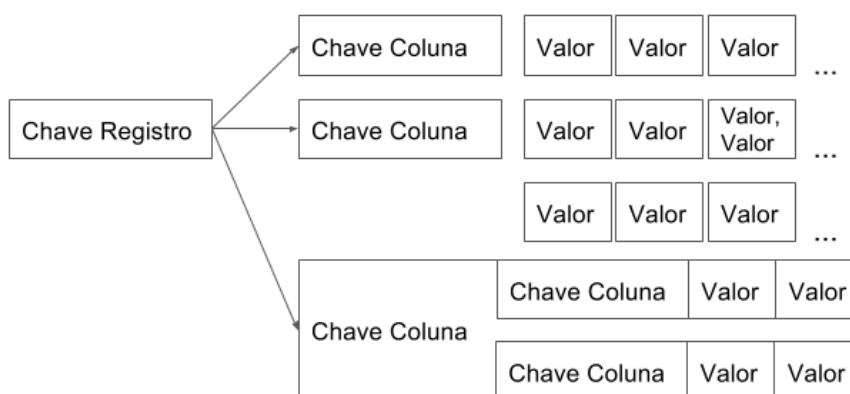
Estrutura de armazenamento colunar:

1 : 123 , 2 : 124;
1 : " Jao da Silva" , 2 : "Maria da Silva" ;
1 : 33 , 2 : 3 1 ;

Temos o Cassandra como exemplo de NoSQL colunar. Este contém um tipo específico de chaveamento, as chaves de partição. Ao contrário do modelo colunar tradicional, a divisão dos arquivos é realizada, através dos campos pertencentes a chave de particionamento (HEWITT, 2016).

As colunas de bases de dados da categoria colunar podem ser definidas em um estilo chamado "Super-Coluna", em que as colunas são inseridas em outras colunas de forma serializada, conforme demonstrado na Figura 8.

Figura 8 - Exemplificação: Colunar



Fonte: autoria própria.

4.3 Documentos

A célula é o bloco básico de construção das estruturas de dados, para seu melhor entendimento pode-se utilizar a analogia, “Podemos imaginar uma célula como uma caixa que é capaz de conter um valor” (AHO ALFRED V.; HOPCROFT, 1983). Em sua implementação, uma célula é utilizada uma chave para endereçar e recuperar o seu valor. Estas células podem ser encadeadas para realizar a composição de matrizes. Outra forma de compor os dados é através de registros. Um registro é uma tupla criada a partir de uma coleção de células chamadas de campos. O registro se difere de uma simples matriz por permitir em sua composição diferentes tipos de dados possibilitando composições de dados heterogêneas. Um exemplo prático de estruturas do tipo registro é o *struct* da linguagem C. A seguir o exemplo:

```
struct Endereço
{
    char destinatario[50];
    char rua[80];
    int numero;
    char complemento[120];
};
```

Temos como exemplos de bases de dados orientadas a documentos o MongoDB e CouchDB¹⁴. O termo documento no contexto do NoSQL não implica em uma estrutura de texto. Um documento é um conjunto de pares chave-valor, cada um podendo ser um item escalar simples ou um elemento composto, tal como uma lista ou documento aninhado (SHARP et al., 2013). Os dados nos campos de um documento podem ser codificados em uma variedade de maneiras, incluindo JSON, BSON, XML, YAML ou mesmo armazenadas como texto simples.

Podemos realizar a equivalência entre um documento e um registro observando um equivalente em JSON:

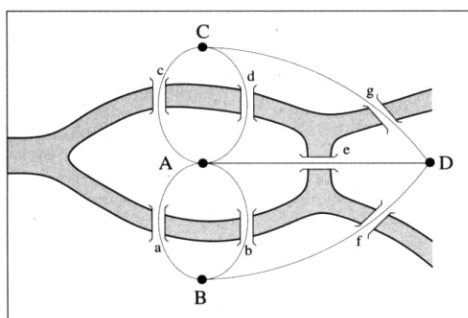
```
endereco: {
  destinatario : " ",
  rua : " ",
  numero : 0 ,
  complemento : " "
}
```

¹⁴ Disponível em: <http://couchdb.apache.org/>. Acesso em: 01 maio 2017.

4.4 Grafos

Um grafo é um ramo da matemática onde relações e objetos de um determinado conjunto são modelados em vértices e arestas $G = (V, A)$ (CORMEN et al., 2001). Nota-se que referências e documentações sobre o NoSQL utilizam a terminologia “nó”¹⁵, para se referir a vértices e “relação” para arestas – essa mesma terminologia será utilizada neste trabalho. A teoria dos grafos foi criada por Euler para a solução de um problema prático. Em 1736 havia a discussão na cidade de Königsberg se era possível atravessar todas as pontes da cidade sem repetir nenhuma, conforme observado na Figura 9.

Figura 9 - Sete Pontes de Königsberg



O layout de Königsberg antes de 1875, com a ilha de Kneiphof (A) e a faixa de terra D encravada entre as duas vertentes do rio Pregel. Fonte: (BARABASI, 2002).

Para solucionar o problema de Königsberg, foi preciso encontrar um caminho em volta da cidade que fizesse uma pessoa cruzar cada ponte apenas uma vez. Em 1736, Leonard Euler inventou a teoria dos grafos ao substituir cada uma das quatro faixas de terra por nós (A a D) e cada ponte por uma aresta (a a g), obtendo um grafo formado por quatro nós e sete arestas. A partir do grafo de Königsberg, ele provou que não há uma rota que cruze cada aresta apenas uma vez (BARABASI, 2002, p.10).

A lógica de Grafos é utilizada para representar listas e matrizes para a realização de algoritmos, como por exemplo encontrar o melhor caminho. Árvores binárias e listas encadeadas são exemplos clássicos de estruturas que utilizam

¹⁵ A mesma terminologia utilizada para a descrição de *Clusters*, sendo que estes são baseados em grafos, em que os nós são os servidores e as arestas são as conexões de rede entre eles.

grafos (CORMEN et al., 2001). Este tipo de lógica é a base de sistemas como o de GPS (*Global Positioning System*) e de redes sociais.

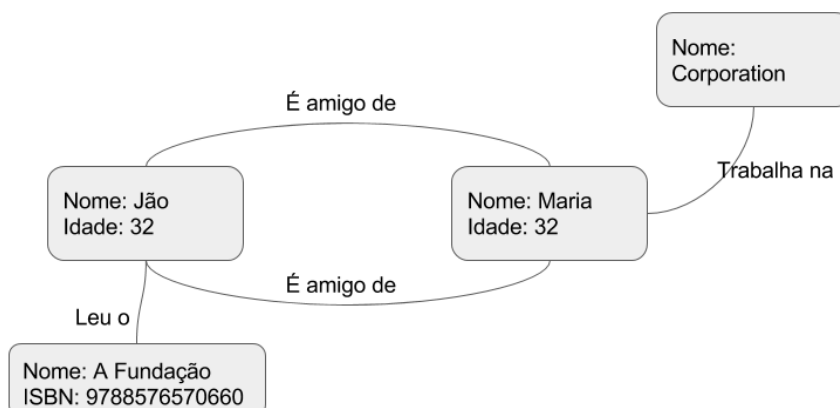
Bases fundamentadas na estrutura de grafos são uma exceção dentro do ecossistema do NoSQL. Sua singularidade vem do fato de que são a única categoria que não é orientada a consultas. Estes modelos são motivados por uma diferente necessidade, pequenos registros com interconexões complexas (SADALAGE; FOWLER, 2012). São exemplos de bases orientadas a grafos: Neo4J; InfiniteGraph¹⁶; HyperGraphDb¹⁷. A constituição dos objetos no Neo4J é realizada através do uso de predicados que definem um nó (sujeito) e suas propriedades (ROBINSON; WEBBER; EIFREM, 2015). Sua principal diferença em relação ao modelo relacional é que não são utilizadas chaves estrangeiras para realizar os relacionamentos. A definição das relações é criada utilizando um *Verbo* que conecte os objetos como pode ser observado na Figura 10. Os objetos utilizados para a composição da base de dados Neo4J são descritos da seguinte forma (NEO TECHNOLOGY, 2016):

- Propriedades: pares de chave-valor contidos nos objetos do modelo;
- Nós: representam entidades como pessoas, empresas, contas ou qualquer outro item a ser rastreado. Eles representam as tuplas contidas na base de dados.
- Relações: são as linhas que conectam nós a outros nós, representando as relações entre eles. São utilizadas para identificar padrões ao examinar as conexões e interconexões entre os nós. Relações contêm propriedades que representam a forma que os nós se relacionam.
- Rótulos: utilizado para agrupar nós em conjuntos, sendo que todos os nós rotulados com o mesmo marcador pertencem ao mesmo conjunto. Operações de banco de dados podem trabalhar com esses conjuntos em vez de todo o grafo, tornando-as mais eficientes. Rótulos são uma adição opcional; desta forma um nó pode conter zero ou mais rótulos.

¹⁶ Disponível em: <http://www.objectivity.com/products/infinitegraph/>. Acesso em: 01 set. 2016.

¹⁷ Disponível em: <http://hypergraphdb.org/>. Acesso em: 01 set. 2016.

Figura 10 - Exemplificação: Modelo orientado a grafos



Fonte: autoria própria.

Bases de dados orientadas a grafos não são soluções novas, sendo citadas sem comparativo direto por Codd (1970) em seu artigo original sobre o modelo relacional. Como exemplos podemos citar as bases IMS da IBM, que na década de 1960 suportavam estruturas hierárquicas baseadas em grafos (IBM, 2011).

4.5 Modelagem

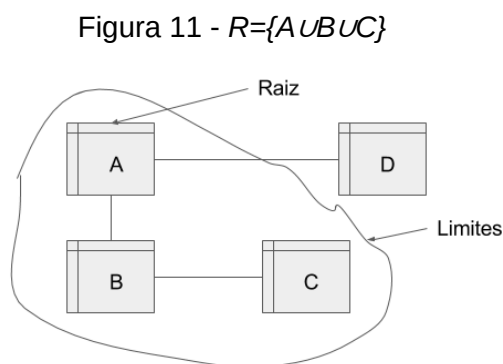
A modelagem relacional costuma ser conduzida pela estrutura dos dados disponíveis, descrevendo os domínios (entidades) utilizados e suas relações. A modelagem relacional inclui em seu processo a criação dos artefatos, modelo conceitual, lógico e físico (BADIA; LEMIRE, 2011): (I) Modelagem conceitual, mapeia as entidades e relações entre elas; (II) Modelagem lógica, realiza a normalização dos dados e (III) Modelagem física, implementa o modelo na base de dados destino. A sua constituição é direcionada a quais informações o modelo irá conter (KATSOV, 2012). Em comparação, a modelagem de dados NoSQL é iniciada a partir das consultas específicas da aplicação. A construção do seu modelo é direcionada a quais "perguntas" o modelo irá responder (KATSOV, 2012). Portanto, bases de dados NoSQL são orientadas para consulta, sua modelagem é concentrada nas necessidades da aplicação e diretamente construída para a manipulação dos dados (VIEIRA et al., 2012; SADALAGE; FOWLER, 2012).

Vários autores observaram a necessidade do desenvolvimento de metodologias e ferramentas que suportam o projeto de banco de dados NoSQL

(BUGIOTTI et al., 2014). Atualmente, a modelagem de banco de dados para sistemas NoSQL é baseada em boas práticas e orientações, atreladas especificamente à ferramenta utilizada (KATSOV, 2012). Entre estas práticas, a utilização dos Agregados.

4.5.1 Agregados

O Agregado é definido como um conjunto de objetos associados que são tratados como uma unidade para a realização de operações (EVANS, 2004). Cada agregado é definido por: (I) a raiz é o objeto central, sendo que todos os objetos contidos fazem referência a ele; a raiz é o único objeto que pode conter referências externas (II) o limite circunscreve os objetos que fazem parte do agregado. Somente a raiz é visível fora de seus limites, os demais são distinguíveis somente dentro do agregado. Para a utilização de agregados no modelo relacional se faz necessário o agrupamento de duas ou mais entidades. Conforme demonstrado na Figura 16, dados conjuntos A, B, C ... N, para a obtenção de um resultado entre A, B e C, seria necessário o acesso aos três conjuntos.



Fonte: autoria própria.

O acesso a duas ou mais entidades no modelo relacional traz a necessidade de operações de Junção. A Junção torna as operações mais onerosas para o processamento, o que faz com que o tempo de resposta seja lento (TAHARA; DIAMOND; ABADI, 2014). A complexidade para a realização dessas operações se torna ainda maior se utilizadas em sistemas distribuídos. Os domínios A e B podem se encontrar em um nó diferente do *cluster*. Isso implica em sacrifícios no projeto do banco de dados, como a desnormalização (EVANS, 2004). Essas desnormalizações no modelo se fazem necessárias para: (I) criar unidades

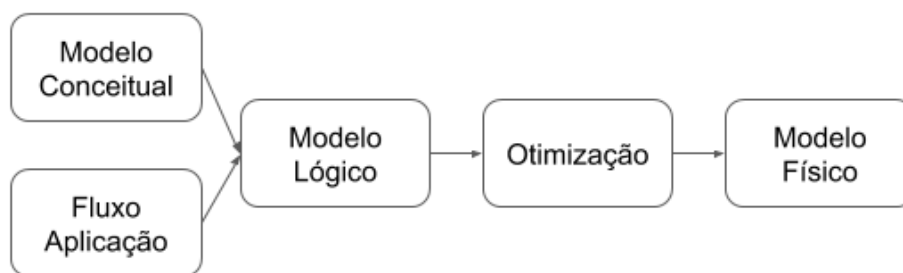
transientes para os dados a serem operados, garantindo menos acesso a disco e (II) utilização de bases criadas para o uso de outras aplicações (EVANS, 2004).

A alta compatibilidade dos Agregados com o NoSQL, faz com que ele seja utilizado em varias propostas de modelagem, como é apresentado a seguir.

4.5.2 Cassandra

Cassandra implementa uma modelagem similar ao fluxo clássico Conceitual, Lógica e Física. À sua modelagem são adicionados passos de mapeamento de fluxo da aplicação e otimização do modelo físico (HEWITT, 2016), conforme pode ser observado na Figura 12.

Figura 12 - Fluxo modelagem Cassandra



Fonte: (HEWITT, 2016)

Inicialmente é estabelecido o mapeamento das entidades e as relações, da mesma forma como proposto por Chen (1975) para o modelo relacional. Em paralelo, são mapeados os fluxos que a aplicação irá realizar para determinar quais as consultas. O modelo lógico construído é resultante do mapeamento de fluxo em conjunto com o modelo conceitual (CHEBOTKO; KASHLEV; LU, 2015). Este mapeamento consiste em definir quais entidades e atributos serão utilizados em cada etapa do fluxo da aplicação. A última etapa consiste em otimizar as consultas obedecendo as necessidades de chaveamento para a implementação do modelo físico.

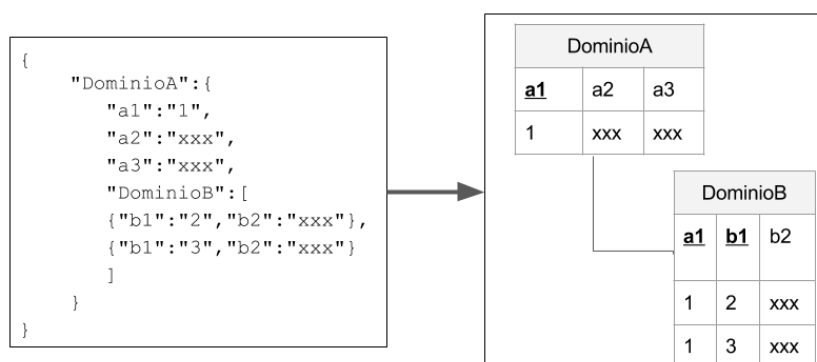
4.5.3 Modelo Entidade Relacionamento para Documentos

Segundo Zhao (2013), às estruturas de documentos do MongoDB podem ser traduzidas em entidades do modelo relacional. Contudo, dada a dinamicidade empregada ao esquema do MongoDB, o esquema em uma coleção pode ser

alterado a qualquer momento. Em tal situação, considerando T o fator tempo, se $T_2 - T_1 > 0$, não podemos determinar o esquema no tempo T_2 , mesmo sabendo o esquema em tempo T_1 (ZHAO et al., 2013).

Os relacionamentos podem ser modelados de duas maneiras: utilizando *Embedded Documents* (Documentos Embutidos) ou os referenciando (VERA et al., 2015). *Embedding documents* são criados inserindo atributos do documento B no documento A , como pode ser visto na Figura 13.

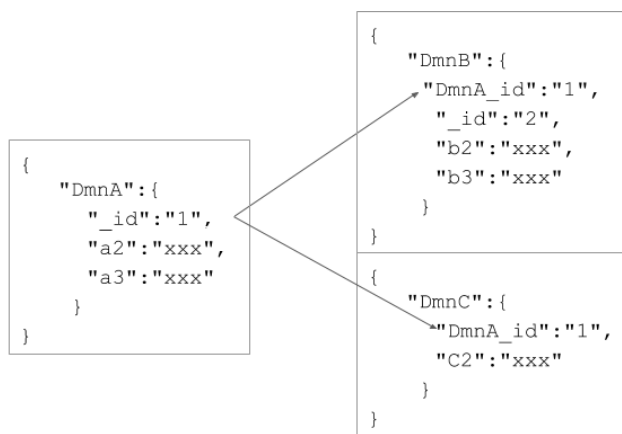
Figura 13 - Documento Embutido



Fonte: autoria própria.

Ao invés de incorporar todos os dados, o documento pode armazenar a chave do documento como referência externa para demonstrar a relação. Documentos referenciados podem ser vistos como um equivalente do modelo relacional, utilizando chaves estrangeiras (ZHAO et al., 2013), conforme pode ser observado na Figura 13.

Figura 14 - Documentos Referenciados



Fonte: autoria própria.

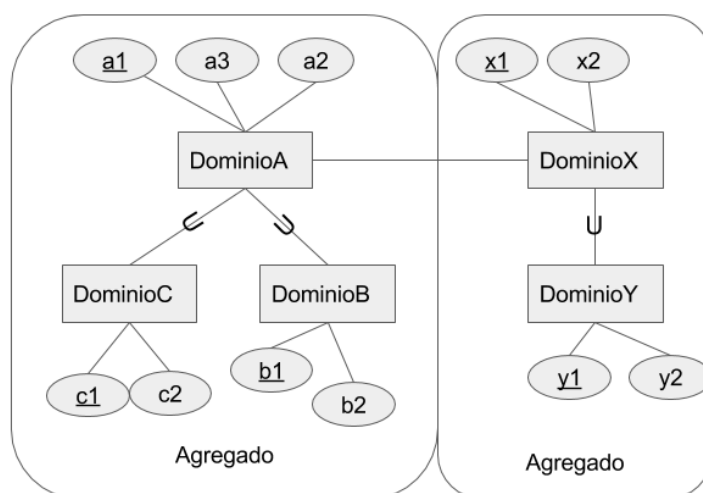
4.5.5 Modelagem EER para NoSQL

O Modelo proposto por Lima (LIMA, 2016), utiliza o modelo Entidade Relacionamento Estendido (EER) para modelar bases da categoria Documentos. O EER inclui todos os conceitos de modelagem propostos por Chen (1975), incluindo as propriedades de subclasse, superclasse e união (também conhecida como categoria). Superclasses e subclasses são utilizadas para explicitar os conceitos de especialização e generalização. Sendo Y uma subclasse de X: $Y \subset X \leftrightarrow \forall \{x \in Y \wedge x \in X\}$

A união é utilizada para representar um conjunto de entidades que combinadas formam diferentes entidades.

A abordagem de Lima utiliza as propriedades da agregação para criar as superclasses e subclasses, estas sendo os limites e raiz do agregado (LIMA, 2016), como pode ser observado na Figura 15:

Figura 16 - EER Agregado



Fonte: Autoria Própria

4.6 Comparativo

Apesar da diferença existente entre os modelos propostos pelas categorias de NoSQL, diversas similaridades são encontradas, principalmente no que tange a estruturação dos dados.

4.6.1 Estruturação

Formatos não estruturados, de acordo com as referências encontradas, são definidos como arquivos. Sua dificuldade não se encontra no armazenamento, mas sim em seu processamento para o aproveitamento e indexação de seu conteúdo. Existem propostas utilizando SGBS NoSQL¹⁸ por conta de suas características para o processamento de NPL (Processamento de Linguagem Natural) para conseguir este conteúdo. Contudo, podem ser armazenados em uma base de dados em um formato de chave-valor comum. Formatos semiestruturados se concentram em dados armazenados de forma serializada.

No ponto de vista teórico, dados da *web*, sensores, documentos serializados como XML e JSON, podem ser expressados em qualquer formato, como grafos, tabelas e objetos (TOTH, 2008). Eles podem ser expressados em documentos, em chaves-valor aninhadas, super-colunas e em nós relacionados. Utilizando esta afirmação, consideremos que as diferenças entre os NoSQL se encontram no metamodelo e operações.

4.6.2 Metamodelo

Consideremos a categoria documentos como a mais simples dentre os NoSQL, realizando através do tipo de dado Registro o comparativo com os demais. A divisa entre os bancos de dados chave-valor e documentos é tênue. Ambos não necessitam de um esquema de dados definidos, qualquer inserção realizada será aceita pela base. Os bancos de dados chave-valor armazenam seu conteúdo em registros similares aos documentos (SADALAGE; FOWLER, 2012).

Dicionários são cadeias de registros acessados por sua chave. Sua diferenciação fica mais clara quando observada a forma de acesso aos dados. No armazenamento de chave-valor só podemos acessar um agregado por pesquisa com base em sua chave. O banco de dados orientado a documento nos permite maior flexibilidade, possibilitando operações com base nos campos no agregado

¹⁸ Disponível em: <http://blog.cloudera.com/blog/2010/03/natural-language-processing-with-hadoop-andpython/>. Acesso em: 10 jan. 2017.

(SADALAGE; FOWLER, 2012). A utilização de indexadores, criando chaves para acessos a documentos podem solucionar.

O colunar pode ser visto como uma lista de registros com alta indexação de chaves (HEWITT, 2016). Estruturalmente, sua diferença com a categoria documentos é a necessidade de um esquema definido. O nível de aninhamento possível na serialização oferecido se limita à super-coluna, enquanto no chave-valor e documentos pode ser estendido indefinidamente, como um JSON ou XML. O Cassandra permite o acesso aos dados somente pela chave.

Para o acesso por demais campos em bancos de dados chaves-valor e colunar, podem ser criados registros auxiliares para a pesquisa (HEWITT, 2016; AEROSPIKE, 2017), conforme observado no exemplo a seguir, em que são descritos produtos e abaixo criados auxiliares para realizar sua busca.

```
produtos:
{ _id: "123", nome: "Carrinho de mão", categoria: ["Jardim", "Ferramenta"]}
{ _id: "737", nome: "Bomba de pneu", categoria: ["Ferramenta"]}

pesquisas por categorias:
{key: "categoria/Ferramenta", produtos: [ "123", "737" ]}
{key: "categoria/Jardim", produtos: [ "123" ]}
```

Mesmo com as semelhanças apresentadas, entre as diferentes categorias de NoSQL. As estruturas presentes por cada categoria, propiciam diferentes operações a serem realizadas com os dados.

4.7 Operações Com Dados

A seguir são demonstradas as operações realizadas em SGBDs NoSQL, utilizando como comparativo direto as operações definidas por Codd, no modelo relacional. Em seguida, são demonstradas as mesmas operações realizadas em SGBDs NoSQL.

Segundo Codd (1970), operações com dados podem ser realizadas utilizando álgebra relacional, um derivado da lógica de primeira ordem. Em SGBDs relacionais é utilizada a linguagem SQL (*Structured Query Language*)¹⁹, que realiza uma alta abstração destas operações, tornando fácil seu manuseio. É a linguagem definida para a realização das operações da álgebra relacional em bases de dados

¹⁹ Para meios de exemplificação, foi utilizado o padrão encontrado em (ISO/IEC..., 2000).

relacionais. De acordo com Date (2010), as operações para realizar a manipulação de dados são: Seleção, Projeção, União, Junção e Diferença.

- Seleção: Representada pela letra grega sigma (σ), a operação restringe os elementos do conjunto que atendam as expressões indicadas no predicado p . O retorno é um subconjunto dos elementos que atendam ao predicado.

$$\sigma(C) = \{x \mid p(x) \wedge x \in C\}$$

Em SQL a operação é criada com a expressão indicada na cláusula *WHERE*. A execução retorna as tuplas que atendam a condição p expressada.

```
SELECT * FROM C WHERE Atributo = p;
```

- Projeção: representada pela letra grega pi (π), a expressão restringe os elementos que serão retornados no conjunto resultante.

$$\pi_{e1,e2,...,en}(C) = \{x[e1, e2, ... en]: x \in C\}$$

A operação em SQL restringe as chaves que serão retornadas de uma tabela, sendo o resultado da operação o conjunto de tuplas com os valores referentes às chaves.

```
SELECT e1,e2....en FROM C;
```

- União: representada pelo símbolo matemático união ($\cup$), a União cria um novo conjunto, sendo este a soma de todos elementos e $C1$ e $C2$.

$$C1 \cup C2 = \{x \mid x \in C1 \vee x \in C2\}$$

Em SQL a operação retorna todas as tuplas presentes nas tabelas $C1$ e $C2$ consultadas.

```
SELECT * FROM C1, C2;
```

- Junção: representada pelo símbolo $\bowtie$, a diferença cria um conjunto proveniente da relação entre dois ou mais conjuntos. Esta relação é expressa pela equivalência dos elementos presentes nos conjuntos relacionados.

$$C1 \bowtie C2 = \{x \cup y \mid x \in C1 \wedge y \in C2 \wedge f(x \cup y)\}$$

Em SQL a expressão indica quais as chaves serão utilizadas para realizar a equivalência entre as tabelas. O resultante são as tuplas C1 e C2, onde as chaves C2.y contenham as chaves de C1.x.

```
SELECT * FROM C1 INNER JOIN C2 ON C1.x = C2.y;
```

- Diferença: representada pelo símbolo matemático menos (-), demonstra uma relação de diferença entre os conjuntos, sendo esta relação expressa por elementos existentes no conjunto C1 e não existentes no conjunto C2.

$$C1 - C2 = \{x | x \in C1 \wedge x \notin C2\}$$

Em SQL a expressão indica quais as chaves serão utilizadas para realizar a diferença entre as tabelas. O resultante são as tuplas C1 e C2, em que as chaves C2.X não contenham as chaves de C1.X.

```
SELECT * FROM C1, C2 WHERE C1.x <> C2.x;
```

4.7.1 Aerospike

O Aerospike utiliza uma linguagem similar ao SQL, o AQL (*Aerospike Query Language*) (AEROSPIKE, 2017), sendo as operações de seleção e projeção idênticas ao SQL. *Namespaces* são contêineres de dados de nível superior. Um *namespace* pode realmente fazer parte de um banco de dados ou um grupo de bancos de dados, como referido no RDBMS padrão. A forma como você coleta dados em *namespaces* se relaciona com, como os dados são armazenados e gerenciados. Todas as operações executadas, utilizam o formato <namespace>.colecao.

```
SELECT * FROM <namespace>.colecao;
```

Os bancos de dados orientados a chave-valor são agnósticos ao conteúdo dos persistidos dos seus registros pela utilização de *Hash* para o armazenamento. Por essa característica, as operações de União, Junção e Diferença são impossibilitadas. Operações mais complexas podem ser desenvolvidas criando funções UDFs (*User Defined Function*) (AEROSPIKE, 2017). Em sua execução, os dados são levados à memória volátil para seu tratamento.

4.7.2 Cassandra

O Cassandra utiliza uma linguagem derivada do SQL a CQL (*Cassandra Query Language*) (APACHE, 2012). Suas operações de Seleção e Projeção são idênticas às realizados em SQL. Contudo não existe suporte às operações de União, Junção e Diferença. O operador de seleção encontra alguns delimitadores em comparação com SGDBs relacionais, as operações de seleção são restritas as chaves primárias ou de partição. Assim, cada necessidade da aplicação deverá ser levada em conta em sua modelagem, para a criação das chaves.

O Cassandra oferece em sua biblioteca a possibilidade da criação de funções UDF (*User Definied Function*) e UDA (*User Defined Aggregate*) (APACHE, 2012). O Cassandra suporta funções escritas em Java, *JavaScript*, *Groovy*, *Scala*, *JRuby* e *JPython*.

As operações em UDA são compostas por duas funções UDF, uma “função de estado” e “uma função final” (APACHE, 2012). A função de estado chama incrementalmente todas as linhas, o resultado de cada chamada é o novo estado. Depois que todas as linhas foram processadas pela função de estado, a função final é chamada com o último valor de estado como seu parâmetro e retorna o valor agregado.

4.7.3 MongoDB

O MongoDB tem uma vasta possibilidade de operações para manutenção dos dados, sendo que somente a operação de diferença não é encontrada em sua biblioteca (ZHAO et al., 2013). A sintaxe básica para operações no MongoDB é `db.colecao.comando()`. As operações de projeção e seleção podem ser realizadas conforme descrito a seguir, onde p é a condição e a são os atributos para a projeção.

Seleção:

```
db.collection.find({p},{a1:1,a1:1..an:1})
```

Representada como: $f(C, p, \{a1, a2, \dots an\}) = \pi_{a1, a2, \dots an}(\sigma(C))$

Operações como União e Junção podem ser realizadas utilizando agregações ou *MapReduce*. As operações de agregação podem ser feitas

isoladamente ou de forma serializada e incremental, sendo que na serialização as transformações são realizadas à medida que as operações são executadas e cada saída de uma operação serve como entrada para a seguinte (MONGODB, 2017b). A operação final produz um documento como resultante.

```
db.orders.aggregate ({
  União -> {$union{arg}}
  Junção -> {$lookup{arg}}
  Diferença -> { $setDifference {arg}}
})
```

Operações mais complexas podem ser criadas utilizando o Framework *MapReduce* do MongoDB (MONGODB, 2017b). Operações criadas desta forma têm maior flexibilidade para a realização de operações, permitindo escrita de funções em *JavaScript*.

```
db.orders . aggregate ({
  map -> function (){}
  reduce -> function (){}
})
```

4.7.4 Neo4J

O Neo4J utiliza a linguagem *Cypher*²⁰ para a realização de suas operações (NEO TECHNOLOGY, 2016b). Por se tratar de uma linguagem que lida com relações de conjuntos, todas as operações da álgebra relacional são aplicadas ao *Cypher*. A sintaxe básica do *Cypher*, *MATCH*, define quais os objetos serão manipulados pela operação (rótulos, nós ou relações), *WHERE* realiza a seleção e *RETURN* define a projeção. A principal diferença entre a linguagem SQL e o *Cypher* é que a cláusula *WHERE* só realiza a seleção nos objetos referenciados na cláusula *MATCH*.

Seleção e Projeção:

```
MATCH (n)
WHERE n.Atributo = p
RETURN n.Atributo
```

²⁰ Disponível em: <http://www.opencypher.org/>. Acesso em: 01 maio 2017.

A operação de junção é definida utilizando os nós por meio de propriedades ou rótulos e exprimindo a cláusula “-->” entre eles. O relacionamento desejado pode ser expresso para explicitar qual conexão entre os nós será utilizada. No exemplo utilizado a seguir são retornadas todas as pessoas que "Jão" conhece.

Junção:

```
MATCH (n:jão)-[:CONHECE]->(nRelacionado)
RETURN n, nRelacionado
```

Para a realização da operação de diferença são selecionados os nós e o operador <> é adicionado na cláusula *WHERE* para o comparativo. O Neo4j possibilita a avaliação de rotas entre nós por meio de operação, executando um algoritmo de busca bidirecional e avaliando diferentes composições de predicados (ROBINSON; WEBBER; EIFREM, 2015). A realização desta prática se torna mais eficiente quando realizada em um conjunto de objetos que atendam ao mesmo predicado (NEO TECHNOLOGY, 2017). Por exemplo, ao procurar o caminho mais curto onde todos os nós têm o rótulo "Pessoa" ou onde não há nós com uma propriedade de "nome". No exemplo a seguir, dado um grupo de pessoas, será demonstrado a rota mais curta que relaciona "Jão" com "Maria":

```
MATCH p= rotacurta (
(n: Pessoa {nome : " Jão"})-[*]-(m: Pessoa {nome : " Maria "})
)
RETURN p
```

5. MODELOS DE PERSISTÊNCIA

Este capítulo apresenta a evolução das técnicas de persistência de dados ao longo das décadas demonstrando como esta evolução nos leva à situação atual com a emergência de tecnologias de persistência não relacionais.

5.1 Persistência Relacional

Na década de 1970/1980, muitas corporações iniciaram a adoção pela computação para o processamento de informações. Porém, era comum que o processamento a ser realizado excedesse a capacidade da memória principal (AHO ALFRED V.; HOPCROFT, 1983). Técnicas de persistência como para a utilização de memória secundária eram implementadas por recursos presentes nas linguagens de programação. Essas continham o tipo de dados arquivo, que se destinava a armazenar dados em memória secundária (AHO ALFRED V.; HOPCROFT, 1983). Outra forma de armazenamento de arquivo era a utilização de formatos de arquivos presentes no sistema operacional.

A utilização de arquivos não demonstrou ser eficiente, pois existia uma significativa urgência nos casos de: (I) metodologia unificada para o design de arquivos e bancos de dados passíveis para a utilização em sistemas de arquivos divergentes (II) incorporação semântica de dados, incluindo regras de negócios para interpretação requisitos e especificações de projeto (CHEN, 2015).

No final da década de 1980, a ANSI(*American National Standards Institute*), adotou o MER oficialmente para a documentação de modelos de dados. Também foi iniciado o desenvolvimento de ferramentas CASE(*Computer-Aided Software Engineering*), utilizadas para a modelagem e diagramação para bases de dados. Isso possibilitou a análise dos requisitos de *software* e diagramação dos mesmos, apoiados pelas metodologias desenvolvidas por Chen e Codd (CHEN, 2015).

Na década de 1990 o surgimento e crescimento das linguagens de programação orientadas a objetos levaram a novos desafios. A incompatibilidade entre o modelo relacional e o modelo orientado a objetos (SADALAGE; FOWLER, 2012) foi denominada *Impedance Mismatch* (COPELAND; MAIER, 1984) que é tratada de “impedância” neste trabalho.

5.1.1 Impedância Objeto-Relacional

Os problemas de impedância são categorizados por incompatibilidades estruturais existentes entre os modelos Objeto-Relacional. Um modelo orientado a objetos é uma rede de interação entre objetos discretos e seu acesso é realizado através de navegação. O modelo relacional é declarativo e seu acesso é realizado por conjuntos (COPELAND; MAIER, 1984). A linguagem SQL não fornece uma analogia direta para as estruturas hierárquicas, como o polimorfismo, utilizado no modelo orientado a objetos.

Esse cenário levou ao desenvolvimento de *frameworks* para o seu tratamento, que empregavam técnicas de ORM (*Object-Relational Mapping*) para realizar o desenvolvimento de *software* (IRELAND et al., 2009). O ORM consiste na realização do mapeamento direto das entidades presentes na base de dados para os objetos do modelo. A impedância entre a estrutura de dados no banco de dados e os modelos utilizados pela maioria das aplicações têm um efeito adverso na produtividade dos desenvolvedores. Ao invés de concentrar seu tempo para o tratamento da lógica de negócio, muito esforço era disposto em ORM. Cerca de 25% do esforço para o desenvolvimento do *software* é dispendido neste tratamento (IRELAND et al., 2009).

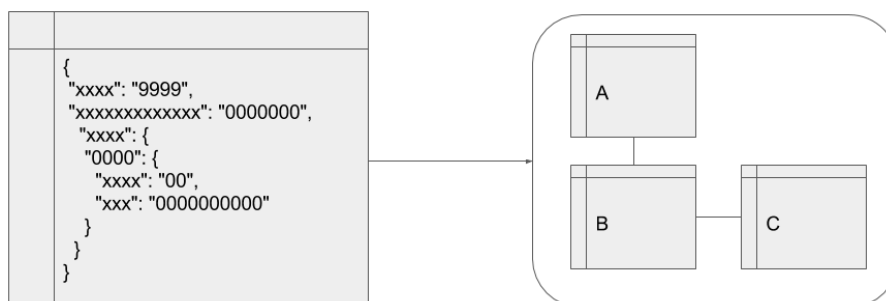
. Na década de 2000, Eric Evans criou o DDD (*Domain-Driven Design*), um conjunto de técnicas e boas práticas para a análise e desenvolvimento de *softwares*. Dentre essas técnicas, os Agregados definem algumas limitações na normalização, que se assumidas facilitam o processo ORM.

5.2 Bases Orientadas a Agregação

A utilização da agregação em bases de dados relacionais leva a quebra de seus princípios inicialmente estabelecidos, tornando-se uma adaptação de conceitos para que um fim seja atingido. Conforme abordado no item 3.2, bases NoSQL são orientadas à operabilidade das mesmas ou modelos orientados à consulta. A unidade de armazenamento em disco será a mesma utilizada para a realização da operação (KATSOV, 2012), conforme pode ser observado na Figura 17, onde somente um registro é utilizado. Esta definição abrange o funcionamento dos bancos de dados de chave-valor, documentos e colunar (SADALAGE;

FOWLER, 2012). A modelagem realizada desta forma vai diretamente ao encontro à implementação do código, tonando o processo de desenvolvimento mais simples (SADALAGE; FOWLER, 2012).

Figura 17 - Agregação, apenas um conjunto $R = \{A\}$



Fonte: autoria própria.

5.3 Bases Orientadas a Grafos

Bases orientadas a grafos não se utilizam dos agregados para a sua estruturação. Seu modelo matemático permite maior flexibilidade. Qualquer estrutura existente no mundo real, como a organização de setores de uma empresa e a estrutura presente no DNA, podem ser representados por ele (RODRIGUEZ, 2017). Os bancos de dados gráficos permitem o armazenamento de dados em seu formato natural, reduzindo o problema de incompatibilidade de impedância (KAUR; RANI, 2013). Contudo, o suporte à Hierarquia não é presente em todas as bases nesta categoria. O Neo4J utiliza OGM (*Object-Graph Mapping*), um *framework* para o mapeamento entre o modelo orientado a objetos e o modelo de persistência em grafos (NEO TECHNOLOGY, 2016a).

5.4 Bancos de dados em Memória Primária

A presente década é marcada por aplicações baseadas em Computação em Nuvem (VIEIRA et al., 2012) com menos restrição de máquina e soluções em larga escala. Este cenário tornou possível o crescimento de bancos de dados com armazenamento em memória volátil. Estas não são um tópico recente. Na década de 1980, em que os “supercomputadores” continham 1,5 MB de memória, propostas como o IBM - *IMS Fast Path* (DEWITT et al., 1984), já eram utilizadas estratégias de armazenamento em memória volátil. Atualmente o poder de

escalabilidade dos sistemas distribuídos e a alocação de memória podem passar de 1Tb (VIEIRA et al., 2012).

Contudo, a memória volátil não está sujeita a falhas, a durabilidade dos dados não é garantida por ela. Soluções híbridas são empregadas para que os dados estejam disponíveis em memória volátil, e duráveis através da persistência em memória secundária. Isso significa que, dependendo do banco de dados, uma ou mais tabelas ficam disponíveis em memória volátil, e não o banco de dados como um todo (GILES; DOSHI; VARMAN, 2016). Temos como exemplos de banco de dados em memória o SAP HANA²¹ e OrientDB²², ambos multiparadigma, permitindo a utilização de diferentes estruturas para a composição do modelo de dados.

²¹ Disponível em: <https://www.sap.com/brazil/products/hana.html>. Acesso em: 01 Set. 2016.

²² Disponível em: <http://orientdb.com/orientdb/>. Acesso em: 01 Set. 2016.

6. CONCLUSÃO

Este trabalho entrega o estado da arte das bases de dados não relacionais denominadas de NoSQL apresentando os problemas, técnicas e tecnologias envolvidas no *Big Data*, NoSQL e computação distribuída. O estabelecimento destas questões e suas soluções demonstra que a atual emergência destas bases é impulsionada pelo *Big Data*. O avanço tecnológico e poder computacional trouxeram a possibilidade da recente popularização de persistências divergentes do modelo relacional.

Modelos orientados a agregação e orientados a grafos, anteriormente utilizados em memória primária, de maneira transiente. O comparativo das bases orientadas a agregação demonstra que estas apresentam similaridades na estruturação hierárquica dos dados, mas as diferenças entre os metamodelos influenciam na indexação e operabilidade dos dados. As bases de dados orientadas a grafos abrem um novo leque de possibilidades por permitirem estruturas mais dinâmicas, possibilitando a diminuição dos efeitos de impedância entre as bases de dados e as estruturas transientes.

A utilização da computação distribuída e os efeitos do avanço tecnológico trouxeram a possibilidade de sistemas tolerantes à partição. Porém, os modelos arquiteturais utilizados na tolerância à partição estão sujeitos à baixa consistência nos dados. Desta forma, conforme pode ser observado no enquadramento dos modelos arquiteturais Cliente-Servidor e Ponto-a-Ponto no Teorema CAP. Contudo, modelos de consistência em memória podem ser utilizados para mitigar essas questões.

6.1 Propostas e Necessidade para Trabalhos Futuros

Bases de dados orientadas a agregação necessitam de uma metodologia e modelagens formalizadas. Conforme descrito nas propostas de modelagem, o EER pode ser utilizado corretamente descrevendo a raiz e limites presentes no Agregado. O MER também é utilizado como modelo para a descrição de agregados em forma de documentos, mas encontra problemas conceituais para a representação da chave-valor e colunares, por não conterem propriedades diferentes do modelo relacional apresentado por Codd. Podemos considerar a utilização desta forma de modelagem

para estes casos como sendo uma adaptação, mas não uma aplicação coerente, o que pode levar a incoerências entre a modelagem, lógica/conceitual e o modelo físico implementado.

A determinação entre as categorias do NoSQL necessita de um melhor embasamento acerca dos modelos propostos e sobre quais as necessidades que cada atende, delimitando quais os requisitos que cada uma delas visa atender, observando as estruturas utilizadas em cada uma das categorias e as operações que são por elas possibilitadas.

A modelagem oferecida pelos bancos de dados orientados a grafos herda características provenientes do modelo relacional e os seus relacionamentos são orientados pela determinação direta de predicados. Apesar da coerência implementada por esta forma de modelagem, ela se torna incompatível quando comparada com um modelo de classes. Então se faz necessário um modelo que permita maior compatibilidade entre a orientação a objetos e grafos.

Dificuldades foram encontradas para a realização deste trabalho no que se refere a definições quanto às bases de dados em memória. Diversas siglas e modelos arquiteturais informais foram encontrados, portanto uma correta formalização e entendimento para a evolução destes modelos se faz necessária.

REFERÊNCIAS

ABELLÓ, A. Big data design. In: *Proceedings of the ACM Eighteenth International Workshop on Data Warehousing and OLAP*. New York, NY, USA: ACM, 2015(DOLAP '15), p. 35–38. ISBN 978-1-4503-3785-4.

AEROSPIKE. *ACID Support in Aerospike*. 2014. Disponível em: <<https://www.aerospike.com/docs/architecture/assets/AerospikeACIDSupport.pdf>>. Acesso em: 13 dez. 2016.

AEROSPIKE. *aql*. 2017. Disponível em: <<http://www.aerospike.com/docs/tools/aql>>. Acesso em: 14 jun. 2016.

AGRAWAL, D.; DAS, S.; ABBADI, A. E. Big data and cloud computing: current state and future opportunities. In: *ACM. Proceedings of the 14th International Conference on Extending Database Technology*. [S.l.], 2011. p. 530–533.

AHO ALFRED V.; HOPCROFT, J. E. U. J. *Data Structures and Algorithms*. 1st. ed. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1983. ISBN 0201000237.

ANGLES, R.; GUTIERREZ, C. Survey of graph database models. *ACM Comput. Surv.*, ACM, New York, NY, USA, v. 40, n. 1, p. 1:1–1:39, fev. 2008. ISSN 0360-0300.

APACHE. *The Cassandra Query Language (CQL)*. 2012. Disponível em: <<http://cassandra.apache.org/doc/latest/cql/>>. Acesso em: 07 nov. 2016.

_____. *MapReduce Tutorial*. 2013. Disponível em: <https://hadoop.apache.org/docs/r1.2.1/mapred_tutorial.html>. Acesso em: 21 set. 2016.

BADIA, A.; LEMIRE, D. A call to arms: revisiting database design. *ACM SIGMOD Record*, ACM, v. 40, n. 3, p. 61–69, 2011.

BANSAL, S. K. Towards a semantic extract-transform-load (ETL) framework for big data integration. *IEEE*, jun 2014.

BARABASI, A.-L. *Linked - A Nova Ciência dos Networks*. [S.l.]: Editora Leopardo, 2002. ISBN 9788528906127.

BASHO. *A technical overview of riak kv enterprise*. [S.l.], 2015. Disponível em: <<http://basho.com/wp-content/uploads/2015/04/RiakKV-Enterprise-Technical-Overview-6page.pdf>>. Acesso em: 01/06/2016. BUGIOTTI, F. et al. Database design for nosql systems. p. 223–231, 2014.

CHANG FAY; DEAN, J. G. S. H. W. C. W. D. A. B. M. C. T. F. A. G. R. E. *Bigtable: A distributed storage system for structured data*. USENIX Association, Berkeley, CA, USA, p. 15–15, 2006. Disponível em: <<http://dl.acm.org/citation.cfm?id=1267308.1267323>>. Acesso em: 01 maio 2016.

CHEBOTKO, A.; KASHLEV, A.; LU, S. *A big data modeling methodology for apache Cassandra*. p. 238–245, jun. 2015. ISSN 2379-7703.
CHEN, P. P. *Entity-relationship modeling: historical events, future trends, and lessons learned*. 2015.

CHEN, P. P.-S. The entity-relationship model: Toward a unified view of data. *SIGIR Forum*, ACM, New York, NY, USA, v. 10, n. 3, p. 9–9, dez. 1975. ISSN 0163-5840.

CODD, E. F. A relational model of data for large shared data banks. *Commun. ACM*, ACM, New York, NY, USA, v. 13, n. 6, p. 377–387, jun. 1970. ISSN 0001-0782.

COPELAND, G.; MAIER, D. Making smalltalk a database system. In: *ACM. ACM Sigmod Record*. [S.l.], 1984. v. 14, n. 2, p. 316–325.

CORMEN, T. H. et al. *Introduction to Algorithms*. 2nd. ed. [S.l.]: McGraw-Hill Higher Education, 2001. ISBN 0070131511.

CUZZOCREA, A.; SONG, I.-Y.; DAVIS, K. C. Analytics over large-scale multidimensional data: The big data revolution! In: *Proceedings of the ACM 14th International Workshop on Data Warehousing and OLAP*. New York, NY, USA: ACM, 2011. (DOLAP '11), p. 101–104. ISBN 978-1-4503-0963-9.

DATE, C. J. C.J. *Date's SQL and Relational Theory Master Class*. 2010. Disponível em: <<https://www.safaribooksonline.com/library/view/cj-dates-sql/9781449389659/>>. Acesso em: 01 fev. 2017.

DEAN, J.; GHEMAWAT, S. Mapreduce: simplified data processing on large clusters. *Communications of the ACM*, ACM, v. 51, n. 1, p. 107–113, 2008.

DEWITT, D. J. et al. Implementation techniques for main memory database systems. ACM, New York, NY, USA, p. 1-8, 1984.

EVANS, E. *Domain-driven design: tackling complexity in the heart of software*. [S.l.]: Addison-Wesley Professional, 2004.

FOX, A.; BREWER, E. A. *Harvest, yield, and scalable tolerant systems*. p. 174–178, 1999.

GARLAN, D.; SHAW, M. An introduction to software architecture. *Advances in software engineering and knowledge engineering*, Singapore, v. 1, n. 3.4, 1993.

GIL, D.; SONG, I.-Y. *Modeling and management of big data: Challenges and opportunities*. Elsevier, 2016.

GILES, E.; DOSHI, K.; VARMAN, P. *Persisting in-memory databases using scm*. p. 2981–2990, 2016.

HAERDER, T.; REUTER, A. Principles of transaction-oriented database recovery. *CSUR, Association for Computing Machinery (ACM)*, v. 15, n. 4, p. 287–317, dec 1983.

HEWITT, J. *Cassandra: The Definitive Guide*, 2nd Edition. [S.l.]: O'Reilly Media, Incorporated, 2016. ISBN 9781491933657.

IBM. *IBM100 Icons of Progress - Information Management System*. 2011. Disponível em: <<http://www-03.ibm.com/ibm/history/ibm100/us/en/icons/ibmims/>>. Acesso em: 22 fev. 2017.

IRELAND, C. et al. A classification of object-relational impedance mismatch. In: IEEE. *Advances in Databases, Knowledge, and Data Applications*, 2009.

DBKDA'09. First International Conference on. [S.l.], 2009. p. 36-43. ISO/IEC FCD 9075-1 2000. Geneva, Switzerland, 2000.

KATSOV, I. *NoSQL data modeling techniques*. 2012. Disponível em: <<https://highlyscalable.wordpress.com/2012/03/01/nosql-data-modeling-techniques/>>. Acesso em: 10 abr. 2016.

KAUR, K.; RANI, R. *Modeling and querying data in nosql databases*. p. 1–7, out. 2013.

KIMBALL, R. *The data warehouse lifecycle toolkit: expert methods for designing, developing, and deploying data warehouses*. [S.l.]: John Wiley & Sons, 1998.

KLEIN, J. et al. Performance evaluation of nosql databases: A case study. In: *Proceedings of the 1st Workshop on Performance Analysis of Big Data Systems*. New York, NY, USA: ACM, 2015. (PABS '15), p. 5–10. ISBN 978-1-4503-3338-2.

LANEY, D. *3d data management: Controlling data volume, velocity and variety*. META Group Research Note, v. 6, p. 70, 2001.

LIMA, C. de. *Projeto lógico de bancos de dados NOSQL documento a partir de esquemas conceituais entidade-relacionamento estendido (EER)*. Dissertação (Mestrado) — Universidade Federal de Santa Catarina, 2016.

LIMA, C. de; MELLO, R. dos S. A workload-driven logical design approach for nosql document databases. *ACM*, New York, NY, USA, p. 73:1–73:10, 2015.

MONGODB. *The MongoDB 3.4 Manual*. 2016. Disponível em: <<https://docs.mongodb.com/manual/>>. Acesso em: 22 jun. 2016.

NEO TECHNOLOGY. *Graph Data Modeling Guidelines*. 2016. Disponível em: <<https://neo4j.com/developer/guide-data-modeling/>>. Acesso em: 20 out. 2016.

_____. *Intro to Cypher*. 2016. Disponível em: <<https://neo4j.com/developer/cypher-query-language/>>. Acesso em: 17 out. 2016.

_____. *The Neo4j Developer Manual v3.1*. 2017. Disponível em: <<http://neo4j.com/docs/developer-manual/3.1>>. Acesso em: 01 out. 2016.
 OBERAI'S, B. *Data warehouse: current scenario & challenges ahead*. 2015. Disponível em: <<http://bhavnaober.blogspot.com.br/2015/02/data-warehouse-current-scenario.html>>. Acesso em: 02 out. 2016.

POSPIECH, M.; FELDEN, C. *Towards a big data theory model*. p. 2082–2090, out. 2015.

RÊGO, B. L. *Gestão e Governança de Dados: Promovendo dados como ativo de valor nas empresas*. [S.l.]: Brasport, 2013.

ROBINSON, I.; WEBBER, J.; EIFREM, E. *Graph Databases: New Opportunities for Connected Data*. 2nd. ed. [S.l.]: O'Reilly Media, Inc., 2015. ISBN 1491930896, 9781491930892.

RODRIGUEZ, M. A. *Open problems in the universal graph theory*. Zenodo, maio 2017. Rodriguez, M.A., "Open Problems in the Universal Graph Theory," *GraphDay'17*, 2(2), pages 1–5, San Francisco, CA, June 2017.

SADALAGE, P. J.; FOWLER, M. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. 1st. ed. [S.l.]: Addison-Wesley Professional, 2012. ISBN 0321826620, 9780321826626.

SAGIROGLU, S.; SINANC, D. *Big data: A review*. IEEE, may 2013.

SANOU, B. *ICT in Facts and Figures 2016*. [S.l.], 2016.

SHARP, J. et al. *Data Access for Highly-Scalable Solutions: Using SQL, NoSQL, and Polyglot Persistence*. 1st. ed. [S.l.]: Microsoft patterns & practices, 2013. ISBN 162114030X, 9781621140306.

SILVA, C. A. R. F. O. da. *Data modeling with NoSQL: how, when and why*. Dissertação (Mestrado) - Faculdade de Engenharia da Universidade do Porto, 2012.

SKIENA, S. S. *The Algorithm Design Manual*. 2nd. ed. [S.l.]: Springer Publishing Company, Incorporated, 2008. ISBN 1848000693, 9781848000698.

TAHARA, D.; DIAMOND, T.; ABADI, D. J. Sinew: A SQL system for multi-structured data. In: *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*. New York, NY, USA: ACM, 2014. (SIGMOD '14), p. 815–826. ISBN 978-1-4503-2376-5.

TANENBAUM, A. S.; STEEN, M. V. *Distributed Systems: Principles and Paradigms* (2nd Edition). Upper Saddle River, NJ, USA: Prentice-Hall, Inc., 2006. ISBN 0132392275.

TOTH, D. *Database engineering from the category theory viewpoint*. Databases, Texts, p. 37, 2008.

VERA, H. et al. Data modeling for nosql document-oriented databases. In: *CEUR Workshop Proceedings*. [S.l.: s.n.], 2015. v. 1478, p. 129–135.

VIEIRA, M. R. et al. *Bancos de dados nosql: conceitos, ferramentas, linguagens e estudos de casos no contexto de big data*. Simpósio Brasileiro de Bancos de Dados, 2012.

VOGELS, W. Eventually consistent. *Commun. ACM*, ACM, New York, NY, USA, v. 52, n. 1, p. 40–44, jan. 2009. ISSN 0001-0782.

ZENNARO, M. Intro to big data. In: *União INTERNACIONAL DE Telecomunicações. ITU ASP COE TRAINING ON "Developing the ICT ecosystem to harness IoTs"*. 2016. Disponível em: <<https://www.itu.int/en/ITU-D/Regional-Presence/AsiaPacific/SiteAssets/Pages/Events/2016/Dec-2016-IoT/IoTtraining/BigData%20-Zennaro.pdf>>. Acesso em: 01 dez. 2016.

ZHAO, G. et al. *Modeling mongodb with relational model*. p. 115–121, set. 2013.

GLOSSÁRIO

Analítico: relativo a/ou próprio da matemática que utiliza demonstrações analíticas por meio do cálculo algébrico.

Categoria: conjuntos que partilham características ou propriedades comuns, o que lhes possibilita serem agrupadas sob um mesmo conceito ou concepção genérica.

Dado: sequência de símbolos que representam um fato.

Framework: biblioteca que oferece uma abstração para um conjunto de práticas, facilitando sua implementação.

Informação: conjunto de Dados que levam à interpretação de um fato.

Persistência: estado consistido em memória não volátil; em memória secundária.

Transação: consiste de uma ou mais operações realizadas em um sistema, de maneira atômica.

Transiente: estado volátil, transitório; em memória primária.

Tupla: sequência ordenada que pode conter 1 ou N elementos.